

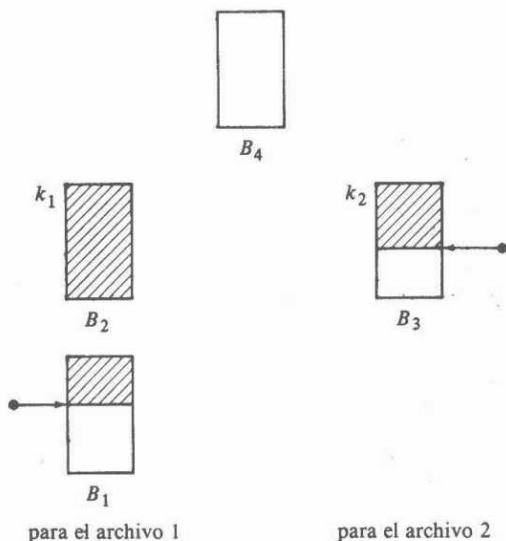


Fig. 11.6. Intercalación con cuatro buffers de entrada.

mento (este es el caso extremo, donde B_1 está vacío y B_3 está lleno en la Fig. 11.6), de modo que la configuración inicial esté segura. En el supuesto de que la figura 11.6 es segura, es necesario mostrar que la configuración lo será después de completar la siguiente transferencia.

Si $k_1 < k_2$, se escoge B_4 para llenarlo con el siguiente bloque del archivo uno y, en otro caso, llenarlo con el del archivo dos. Supóngase primero que $k_1 < k_2$. Ya que B_1 y B_3 de la figura 11.6 tienen exactamente b registros, se debe, durante la siguiente transferencia, agotar B_1 ; de otra forma, es necesario agotar B_3 y contradecir la seguridad de la figura 11.6. Así, después de una transferencia, la configuración se ve como en la figura 11.7(a).

Para comprobar que la figura 11.7(a) es segura, considérense dos casos. Primero, si k_3 , la última clave del bloque recién leído B_4 , es menor que k_2 , entonces, como B_4 está lleno, es muy probable que haya b registros iguales o menores que $\min(k_3, k_2)$, y la configuración es segura. Si $k_2 \leq k_3$, y dado que se supuso que $k_1 < k_2$ (de otro modo se hubiera llenado B_4 del archivo dos), los b registros de B_2 y B_3 tienen claves menores o iguales que $\min(k_2, k_3) = k_2$.

Ahora se estudiará el caso donde $k_1 \geq k_2$ en la figura 11.6. Se escoge leer el siguiente bloque del archivo dos. La figura 11.7(b) muestra la situación resultante. Como en el caso $k_1 < k_2$, se argumenta que B_1 debe agotarse y, por eso, se muestra que el archivo uno tiene asignado sólo el buffer B_2 de la figura 11.7(b). La demostración de que esta figura es segura es igual que la de la figura 11.7(a).

Obsérvese que, como en el esquema con seis buffers, no se lee un archivo más allá del fin de un fragmento. Sin embargo, si no es necesario leer un bloque desde uno de los fragmentos actuales, puede leerse un bloque del siguiente fragmento de ese archivo. Así, existe la oportunidad de leer un bloque de cada uno de los fragmentos siguientes, y entonces será posible iniciar la intercalación de los fragmentos tan pronto como se hayan seleccionado los últimos registros del fragmento anterior.

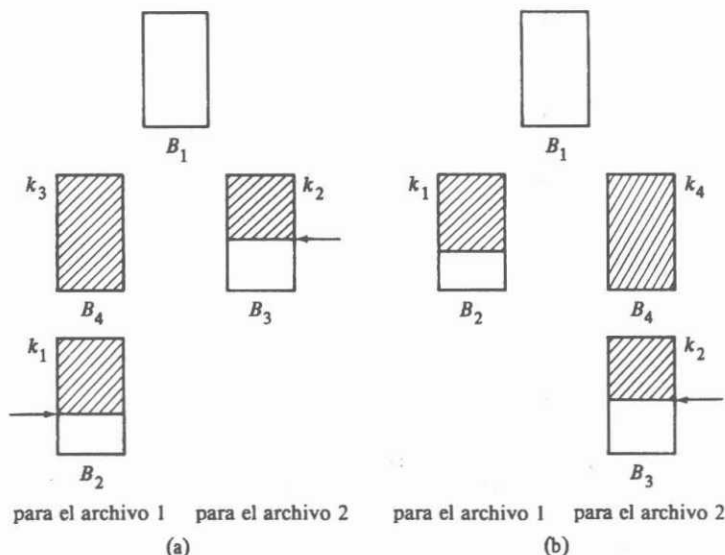


Fig. 11.7. Configuración después de una transferencia.

11.3 Almacenamiento de información en archivos

En esta sección, se analizan las estructuras de datos y los algoritmos para el almacenamiento, y recuperación de información en archivos almacenados en forma externa. Se considerará un archivo como una secuencia de registros, donde cada registro consiste en la misma secuencia de campos. Los campos pueden ser de *longitud fija*, con un número predeterminado de bytes, o de *longitud variable*, con un tamaño arbitrario. Los archivos con registros de longitud fija se suelen utilizar en los sistemas de administración de bases de datos para almacenar datos muy estructurados. Los archivos con registros de longitud variable se usan típicamente para almacenar información de textos; no están disponibles en Pascal. En esta sección, se supondrá

que sólo hay campos de longitud fija; las técnicas empleadas en los registros de longitud fija pueden modificarse con facilidad para trabajar con registros de longitud variable.

Las operaciones con archivos que se considerarán son las siguientes:

1. INSERTA un registro determinado en un archivo en particular.
2. SUPRIME de un archivo en particular todos los registros que tengan un valor asignado en cada campo de conjunto de campos designado.
3. MODIFICA todos los registros de un archivo en particular asignando valores asignados a ciertos campos en los registros que tengan un valor asignado en otro conjunto de campos.
4. RECUPERA todos los registros que tengan valores asignados en cada uno de los campos de un conjunto asignado.

Ejemplo 11.3. Por ejemplo, si se tiene un archivo cuyos registros constan de tres campos: *nombre*, *dirección* y *teléfono*. Se podrá preguntar por todos los registros con *teléfono* = 555-1234, insertar el registro (Juan Pérez, calle Manzana 12, 555-1234) o eliminar todos los registros con *nombre* = «Juan Pérez» y *dirección* = «calle Manzana 12». Como otro ejemplo, se puede desear la modificación de todos los registros con *nombre* = «Juan Pérez» para fijar el campo *teléfono* a 555-1234. □

En buena medida, se pueden considerar las operaciones con archivos como si los archivos fueran conjuntos de registros y las operaciones fueran las que se analizaron en los capítulos 4 y 5. Sin embargo, existen dos diferencias importantes. Primero, cuando se habla de archivos en dispositivos de almacenamiento externo, es forzoso utilizar la medición del costo comentada en la sección 11.1, en la evaluación de las estrategias de organización de archivos. Esto es, se supone que los archivos están almacenados en cierta cantidad de bloques físicos, y que el costo de una operación es el número de bloques que se van a leer en memoria principal o escribir desde memoria principal en el almacenamiento externo.

La segunda diferencia es que los registros, siendo tipos de datos concretos en la mayoría de los lenguajes de programación, puede esperarse que tengan apuntadores a ellos, mientras que los elementos abstractos de un conjunto, normalmente no tendrán «apuntadores» hacia ellos. En particular, los sistemas de bases de datos con frecuencia hacen uso de apuntadores a registros cuando organizan datos. La consecuencia de dichos apuntadores es que los registros suelen considerarse *adheridos*; no pueden moverse por el almacenamiento, debido a la posibilidad de que un apuntador de algún lugar desconocido no consiga apuntar al registro si éste se ha movido.

Una forma simple de representar apuntadores a registros es la siguiente. Cada bloque tiene una *dirección física*, que es la localización del inicio del bloque en el dispositivo de almacenamiento externo; es función del sistema de archivos cuidar las direcciones físicas. Una forma de representar las direcciones de los registros es usar la dirección física del bloque que contiene el registro junto con un *desplazamiento*, que da el número de bytes que preceden en el bloque al principio del registro. Esos pares físicos dirección-desplazamiento pueden almacenarse en campos de tipo «apuntador a registro».

Organización simple

La forma más simple, y también menos eficiente, de realizar las operaciones de archivos anteriores es usar primitivas de lectura y escritura de archivos como las de Pascal. En esta «organización» (que en realidad es una «falta de organización»), los registros pueden almacenarse en cualquier orden. La recuperación de un registro con valores especificados en ciertos campos se efectúa rastreando el archivo y viendo en cada registro si se encuentran los valores especificados. Una inserción en un archivo puede realizarse agregando el registro al final del archivo.

Para la modificación de los registros, se rastrea el archivo y se prueba cada registro para ver si corresponde a los campos designados, y de ser así, se hacen los cambios necesarios en el registro. Una operación de eliminación trabaja casi siempre de la misma forma, pero al encontrar un registro cuyos campos corresponden a los valores requeridos para que la eliminación se lleve a cabo, se debe encontrar la forma de eliminar el registro. Una posibilidad es correr todos los registros siguientes, una posición hacia adelante en sus respectivos bloques, y pasar el primer registro de cada bloque siguiente a la última posición del bloque anterior del archivo. Sin embargo, este enfoque no funciona si los registros están adheridos, porque un apuntador al i -ésimo registro del archivo apuntaría entonces al $(i + 1)$ -ésimo registro.

Si los registros están adheridos, es necesario usar un enfoque distinto. Se marcan de alguna manera los registros eliminados, pero sin mover registros para llenar el hueco, ni insertar un registro nuevo en ese espacio. Así, lógicamente, el registro se elimina, pero su espacio continúa ocupado en el archivo. Esto es necesario para que, si se sigue un apuntador a un registro eliminado, se descubra que el registro apuntado fue eliminado y se tome la acción apropiada, como hacer que el apuntador sea NIL y no se le vuelva a seguir. Dos formas de marcar los registros cuando se eliminan son:

1. Sustituir el registro por algún valor que nunca pueda ser el valor de un registro «real», y cuando se siga un apuntador, suponer que el registro está eliminado si tiene ese valor.
2. Que cada registro tenga un *bit de eliminación*, un solo bit que es 1 en registros que se han eliminado, y 0 en caso contrario.

Aceleración de operaciones con archivos

La desventaja obvia de los archivos secuenciales es que las operaciones son lentas. Cada operación requiere la lectura del archivo completo, y algunos bloques pueden requerir ser reescritos también. Por fortuna, existen organizaciones de archivos que permiten acceder a un registro, leyendo en la memoria principal sólo una pequeña fracción del archivo completo.

Para hacer posible dichas organizaciones, se supone que cada registro de archivo tiene una *clave*, un conjunto de campos que identifica de manera unívoca cada registro. Por ejemplo, el campo *nombre* del archivo *nombre-dirección-teléfono* puede considerarse una clave. Esto es, se puede suponer que no existen simultáneamente

dos registros en el archivo con el mismo valor en el campo *nombre*. La recuperación de un registro dando valores a sus campos clave, es una operación habitual que se realiza con gran facilidad en muchas organizaciones de archivo comunes.

Otro elemento necesario para lograr operaciones rápidas con archivos es la capacidad de acceder a bloques en forma directa, en vez de recorrer en secuencia los bloques que contienen el archivo. Muchas de las estructuras de datos utilizadas para operaciones rápidas con archivos usarán apuntadores a los propios bloques, los cuales son direcciones físicas de los bloques, como se describió antes. Desafortunadamente, no es posible escribir programas en Pascal, ni en muchos otros lenguajes, que se ocupen de los datos en el nivel de bloques físicos y sus direcciones; dichas operaciones se efectúan de ordinario con mandatos del sistema operativo. Sin embargo, se hará una breve descripción informal de la forma de trabajo de las organizaciones que utilizan el acceso directo a bloques.

Archivos con función de dispersión

La dispersión (*hashing*) es una técnica muy utilizada para tener acceso rápido a información almacenada en archivos secundarios. La idea básica es similar a la dispersión abierta estudiada en la sección 4.7. Los registros de un archivo se dividen entre varias *cubetas*, y cada una consiste en una lista enlazada de uno o más bloques de almacenamiento externo. La organización es similar a la presentada en la figura 4.10. Existe una tabla de cubetas que contiene B apuntadores, uno para cada cubeta. En la tabla de cubetas, cada apuntador es la dirección física del primer bloque de la lista enlazada de bloques para esa cubeta.

Las cubetas están numeradas $0, 1, \dots, B - 1$. Una función de dispersión h hace corresponder cada valor de clave con uno de los enteros 0 a $B - 1$. Si x es una clave, $h(x)$ es el número de la cubeta que contiene el registro con la clave x , si tal registro está presente en alguna. Los bloques que forma cada cubeta se encuentran encadenados y forman una lista enlazada. Así, el encabezado del i -ésimo bloque de una cubeta contiene un apuntador a las direcciones físicas del $(i + 1)$ -ésimo bloque. El último bloque de una cubeta contiene un apuntador NIL en su encabezado.

Esta organización se ilustra en la figura 11.8. La principal diferencia entre las figuras 11.8 y 4.10 es que aquí, los elementos almacenados en un bloque de una cubeta no tienen que estar encadenados con apuntadores; sólo los bloques deben estar encadenados.

Si el tamaño de la tabla de cubetas es pequeño, puede quedar almacenada en memoria principal. De otra forma, puede almacenarse en forma secuencial, en tantos bloques como sea necesario. Para buscar el registro con clave x , se calcula $h(x)$, para encontrar el bloque de la tabla de cubetas que contengan el apuntador al primer bloque de la cubeta $h(x)$. Después, se leen los bloques de la cubeta $h(x)$ de manera sucesiva, hasta encontrar el que contenga el registro con la clave x . Si se agotan todos los bloques de la lista enlazada para la cubeta $h(x)$, se concluye que x no es la clave de ningún registro.

Esta estructura es eficiente en operaciones donde se especifican los valores de

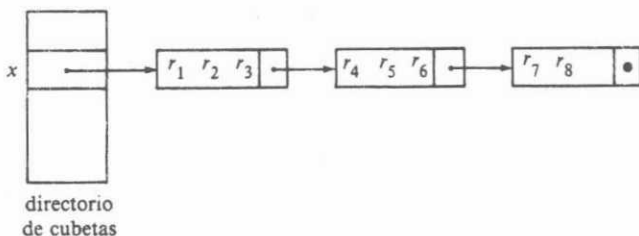


Fig. 11.8. Función de dispersión con cubetas que contienen bloques encadenados.

los campos de la clave, como la recuperación de un registro con un valor especificado en la clave o una inserción de un registro (lo que, como es obvio, especifica el valor de la clave para ese registro). El número promedio de accesos a bloques requerido para una operación que especifica la clave de un registro es aproximadamente el número promedio de bloques en una cubeta, que es n/bk si n es el número de registros, un bloque contiene b registros, y k es el número de cubetas. Así, en promedio, las operaciones basadas en claves son k veces más rápidas con esta organización que con la de archivos no organizados. Lamentablemente, las operaciones no basadas en claves no se aceleran, puesto que se deben examinar esencialmente todas las cubetas durante esas otras operaciones. La única forma general de incrementar la velocidad en operaciones que no se basan en claves es usar índices secundarios, que se analizan en el final de esta sección.

Para insertar un registro con la clave x , primero se verifica si ya existe un registro con la misma clave. De ser así, se informa de un error, dado que se supone que la clave identifica de manera unívoca cada registro. Si no existe el registro con la clave x , se inserta el registro nuevo en el primer bloque de la cadena para la cubeta $h(x)$ en el que pueda caber. Si el registro no cabe en ningún bloque de los existentes para la cubeta $h(x)$, se llama al sistema de archivos para encontrar un bloque nuevo en el cual se pueda colocar el registro. Este bloque nuevo se agrega al final de la cadena de la cubeta $h(x)$.

Para eliminar un registro con clave x , primero debe localizarse, y después se le asigna su bit de eliminación. Otra estrategia posible (que no puede usarse cuando los registros están adheridos) es sustituir el registro eliminado por el último de la cadena de $h(x)$. Si la eliminación del último registro causa el vaciado del último bloque de la cadena se puede devolver el bloque vacío al sistema de archivos para usarlo de nuevo más tarde.

Una organización de archivos de acceso con función de dispersión bien diseñada, requiere sólo unos cuantos accesos a bloques para cada operación de archivo. Si la función de dispersión es buena y el número de cubetas es más o menos igual al número de registros en el archivo, dividido entre el número de registros que pueden caber en un bloque, entonces la cubeta promedio consta de un bloque. Excluyendo el número de accesos a bloques para buscar la tabla de cubetas, una recuperación típica basada en claves hará un solo acceso a un bloque, y una inserción, eliminación o modificación típicas efectuarán dos accesos a bloques. Si el número prome-

dio de registros por cubeta excede en mucho de la cantidad que cabe en un bloque, se puede reorganizar periódicamente la tabla de dispersión incrementando al doble la cantidad de cubetas y dividiendo cada cubeta en dos. Esas ideas se estudiaron al final de la sección 4.8.

Archivos indizados

Otra forma común de organizar un archivo de registros es mantener el archivo clasificado de acuerdo con los valores de las claves. Entonces es posible buscar en el archivo como se haría en un diccionario o en un directorio telefónico, revisando sólo el primer nombre o palabra de cada página. Para facilitar la búsqueda, se crea un segundo archivo, llamado *índice disperso*, que consta de pares (x, b) , donde x es un valor de una clave y b es la dirección física del bloque en el cual el primer registro tiene la clave con valor x . El índice disperso se mantiene clasificado de acuerdo con los valores de las claves.

Ejemplo 11.4. En la figura 11.9, se observa un archivo y su archivo de índice disperso. Se supone que tres registros del archivo principal, o tres pares del archivo de índices, caben en un bloque. Sólo se muestran los valores de las claves de los registros del archivo principal, que se suponen enteros. $\square$

Para recuperar un registro con una clave dada x , primero se busca el par (x, b) en el archivo índice. Lo que en realidad se busca es la z más grande tal que $z \leq x$ y exista un par (z, b) en el archivo índice. Después, la clave x aparece en el bloque b , si está presente en el archivo principal.

Hay varias estrategias para buscar en el archivo de índices; la más simple es la *búsqueda lineal*. Se lee el archivo índice desde el principio hasta encontrar el par (x, b) o el primer par (y, b) , donde $y > x$. En el último caso, el par precedente (z, b') debe tener $z < x$, y si el registro con la clave x está en algún sitio, es en el bloque b' .

La búsqueda lineal sólo es posible para archivos de índices pequeños. Un método más rápido es la *búsqueda binaria*. Supóngase que el archivo de índices está almacenado en los bloques $b_1, b_2, \dots, b_n$. Para buscar la clave x , se toma el bloque medio $b_{\lfloor n/2 \rfloor}$ y se compara x con la clave y del primer par de ese bloque. Si $x < y$, se repite la búsqueda en los bloques $b_1, b_2, \dots, b_{\lfloor n/2 \rfloor - 1}$. Si $x \geq y$, pero x es menor que la clave del bloque $b_{\lfloor n/2 \rfloor + 1}$, (o si $n = 1$ de modo que no hay tal bloque), se utiliza la búsqueda lineal para ver si x corresponde al primer componente de un par de índi-

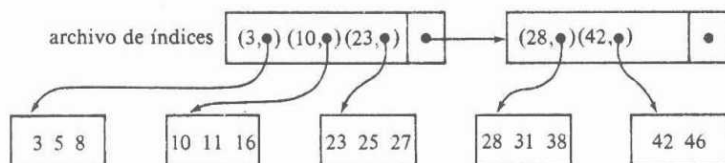


Fig. 11.9. Un archivo principal y su índice disperso.

ces en el bloque $b_{\lfloor n/2 \rfloor}$. De otra forma, se repite la búsqueda en los bloques $b_{\lfloor n/2 \rfloor + 1}$, $b_{\lfloor n/2 \rfloor + 2}$, ..., b_n . Con la búsqueda binaria es necesario examinar sólo $\lceil \log_2(n+1) \rceil$ bloques del archivo índice.

Para asignar valor inicial a un archivo indizado, se clasifican los registros de acuerdo con los valores de sus claves, y después se distribuyen los registros entre los bloques, en ese orden; se puede decidir empaquetar tantos como quepan en cada bloque. Otra posibilidad sería preferir dejar espacio para unos registros adicionales que puedan insertarse más tarde. La ventaja es que después es menos probable que las inserciones sobrecarguen el bloque en el cual la inserción se lleva a cabo, con el consecuente requerimiento de tener acceso a los bloques adyacentes. Después de la división de los registros en los bloques de alguna de esas formas, se crea el archivo de índices examinando cada bloque y encontrando la primera clave de cada bloque. Como en el caso del archivo principal, puede dejarse algún espacio para crecimiento posterior en los bloques que contienen el archivo de índices.

Supóngase que se tiene un archivo clasificado con registros almacenados en los bloques $b_1, b_2, \dots, b_m$. Para insertar un nuevo registro en este archivo, se emplea el archivo de índices para determinar qué bloque b_i debe contener el registro nuevo. Si el registro nuevo debe quedar en b_i , se coloca en el orden adecuado. Después, si el registro nuevo queda como primer registro del bloque b_i , se ajusta el archivo índice.

Si el registro nuevo no cabe en b_i , hay varias estrategias posibles. Tal vez la más simple sea ir al bloque b_{i+1} , el cual puede encontrarse por medio del archivo de índices para ver si el último registro de b_i puede pasarse al principio de b_{i+1} . De ser así, este último registro se pasa a b_{i+1} , y el nuevo puede insertarse en la posición adecuada en b_i . El elemento del archivo de índices correspondiente a b_{i+1} , y posiblemente el de b_i , debe ajustarse en forma apropiada.

Si b_{i+1} también está lleno, o si b_i es el último bloque ($i = m$), entonces se obtiene un bloque nuevo del sistema de archivos. El registro nuevo se inserta en el bloque nuevo a continuación de b_i . Ahora se emplea este mismo procedimiento para insertar un registro que corresponda al nuevo bloque en el archivo de índices.

Archivos no clasificados con índice denso

Otra forma de organizar un archivo de registros es mantener el archivo en orden aleatorio y tener otro, llamado *índice denso*, para ayudar a ubicar los registros. El índice denso consta de pares (x, p) , donde p es un apuntador al registro con la clave x en el archivo principal. Esos pares están clasificados por el valor de la clave, de modo que una estructura como el índice disperso mencionado antes, o el árbol B mencionado en la siguiente sección, puede usarse para ayudar a encontrar las claves en el índice denso.

Con esta organización se emplean los índices densos para encontrar la localización en el archivo principal de un registro con una clave dada. Para insertar un nuevo registro, se sigue la pista del último bloque del archivo principal y ahí se insertan los registros nuevos, tomando un nuevo bloque del sistema de archivos si el último bloque está lleno. También se inserta un apuntador a ese registro en el archivo del

índice denso. Para eliminar un registro, tan sólo se ajusta el bit de eliminación en el registro, y se elimina el elemento correspondiente en el índice denso (tal vez, ajustando también un bit de eliminación).

Índices secundarios

Mientras que las estructuras dispersas e indizadas incrementan bastante la velocidad de las operaciones basadas en claves, ninguna de ellas sirven de ayuda cuando las operaciones implican una búsqueda de registros a partir de valores en campos que no sean campos clave. Si se desea encontrar los registros con valores dados en algún conjunto de campos $F_1, \dots, F_k$ es necesario un *índice secundario* de aquellos campos. Un índice secundario es un archivo formado de pares (v, p) , donde v es una lista de valores, uno para cada uno de los campos $F_1, \dots, F_k$ y p es un apuntador a un registro. Puede haber más de un par con una v dada, y cada apuntador asociado indica un registro del archivo principal que tiene a v como lista de valores de los campos $F_1, \dots, F_k$.

Para recuperar registros dados los valores de los campos $F_1, \dots, F_k$ en el índice secundario se buscan el o los registros con esa lista de valores. El índice secundario mismo puede organizarse en cualquiera de las formas comentadas para la organización de archivos por valores de claves. Esto es, se presume que v sea la clave de (v, p) .

Por ejemplo, una organización con función de dispersión en realidad no depende de que las claves sean únicas, aunque si hubiera muchos registros con la misma «clave», éstos podrían distribuirse en cubetas de manera no uniforme, con el efecto de que la función de dispersión no incrementaría mucho la velocidad de acceso. En el caso extremo, como cuando hay sólo dos valores para los campos de un índice secundario, todas las cubetas excepto dos estarían no vacías, y la tabla de dispersión sólo podría incrementar la velocidad de las operaciones en un factor de dos a lo sumo, sin importar cuántas cubetas haya. De modo semejante, un índice disperso no requiere que las claves sean únicas, pero si no lo son, puede haber dos o más bloques del archivo principal que tengan la misma «clave» menor, y todos esos bloques se recorren al buscar registros con ese valor.

Con cualquier organización, de índice disperso o de índice con función de dispersión para el archivo de índices secundarios, puede desearse ahorrar espacio agrupando todos los registros con el mismo valor. Esto es, los pares $(v, p_1), (v, p_2), \dots, (v, p_m)$ pueden reemplazarse por v seguida de la lista $p_1, p_2, \dots, p_m$.

Cabe preguntarse si el mejor tiempo de respuesta a las operaciones aleatorias no puede obtenerse al crear un índice secundario para cada campo, o aun para todos los subconjuntos de los campos. Lamentablemente, hay un precio por cada índice secundario que se desee crear. Primero, está el espacio necesario para almacenar el índice secundario, y eso puede ser o no un problema, dependiendo de si el espacio es una prioridad.

Además, cada índice secundario creado disminuye la velocidad de todas las inserciones y eliminaciones. La razón es que al insertar un registro se debe insertar también un elemento para ese registro en cada índice secundario, para que los índices

secundarios sigan representando exactamente el archivo. La actualización de un índice secundario requiere por lo menos dos accesos a bloques, ya que se debe leer y escribir un bloque. Sin embargo, puede requerir mucho más de dos accesos a bloques, ya que es necesario encontrar ese bloque, y cualquier organización empleada para el archivo del índice secundario requerirá unos cuantos accesos adicionales, en promedio, para encontrar cualquier bloque. Algo parecido sucede en cada eliminación. La conclusión es que la selección de los índices secundarios requiere seguir un criterio, puesto que debe determinarse qué conjuntos de campos se especificarán en operaciones muy frecuentes de modo que el tiempo ahorrado teniendo esos índices secundarios disponibles compense los costos de la actualización de los índices en cada inserción y eliminación.

11.4 Árboles de búsqueda externa

La estructura de datos tipo árbol presentada en el capítulo 5 para representar conjuntos puede usarse también para representar archivos externos. El árbol B, una generalización de los árboles 2-3 analizados en el capítulo 5, es especialmente adecuada para almacenamiento externo, y se ha convertido en la organización estándar para índices en sistemas de bases de datos. Esta sección presenta las técnicas básicas de recuperación, inserción y eliminación de información en árboles B.

Árboles de búsqueda múltiple

Un árbol de búsqueda m -ario es una generalización del árbol binario de búsqueda en el cual cada nodo tiene como máximo m hijos. Generalizando la propiedad del árbol binario de búsqueda, se requiere que si n_1 y n_2 son dos hijos de algún nodo, y n_1 está a la izquierda de n_2 , los elementos descendientes de n_1 sean todos menores que los de n_2 . Las operaciones MIEMBRO, INSERTA y SUPRIME de un árbol de búsqueda m -ario se realizan con una generalización de las operaciones en árboles binarios de búsqueda, estudiadas en la sección 5.1.

Sin embargo, aquí interesa el almacenamiento de registros en archivos, donde los archivos se depositan en bloques de almacenamiento externo. La adaptación correcta de la idea de árboles múltiples consiste en pensar en los nodos como bloques físicos. Un nodo interior contiene apuntadores a sus m hijos y también contiene $m-1$ claves que separan los descendientes del hijo. Los nodos hojas son bloques también, y contienen los registros del archivo principal.

Si se emplea un árbol binario de búsqueda de n nodos para representar un archivo almacenado en forma externa, pueden requerirse, en promedio, $\log_2 n$ accesos a bloques para recuperar un registro del archivo. En cambio, si se utiliza un árbol de búsqueda m -ario para representar el archivo, requerirá, en promedio, sólo $\log_m n$ accesos a bloques para recuperar un registro. Para $n = 10^6$, el árbol binario de búsqueda puede requerir cerca de 20 accesos a bloques, mientras que un árbol de 128 caminos requerirá sólo 3.

No se puede hacer m arbitrariamente grande, pues cuanto mayor sea m , mayor deberá ser el bloque. Más aún, es más lento leer y procesar un bloque más grande, así que hay un valor óptimo de m que reduce la cantidad de tiempo necesario para recorrer un árbol m -ario de búsqueda externa. En la práctica, se obtiene un valor cercano al mínimo para un amplio intervalo de m . (Véase Ejercicio 11.18.)

Arboles B

Un árbol B es una clase especial de árbol m -ario balanceado que permite recuperar, eliminar e insertar registros de un archivo externo con buen rendimiento en el peor caso. Es una generalización de los árboles 2-3 de la sección 5.4. Formalmente, un árbol B de orden m es un árbol de búsqueda m -ario con las siguientes propiedades:

1. La raíz es una hoja o tiene al menos dos hijos.
2. Cada nodo, excepto la raíz y las hojas, tiene entre $\lceil m/2 \rceil$ y m hijos.
3. Cada camino desde la raíz hasta una hoja tiene la misma longitud.

Obsérvese que todo árbol 2-3 es un árbol B de orden 3. La figura 11.10 muestra un árbol B de orden 5, en el cual se supone que caben como máximo tres registros en un bloque hoja.

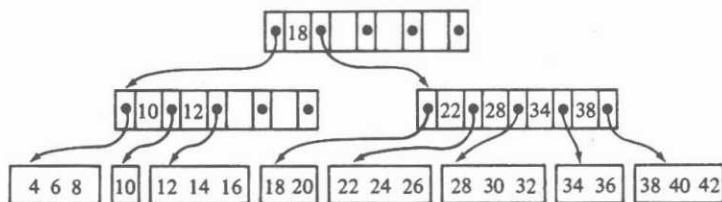


Fig. 11.10. Árbol B de orden 5.

Se puede considerar un árbol B como un índice jerárquico en el cual cada nodo ocupa un bloque en el almacenamiento externo. La raíz del árbol B es el índice de primer nivel. Cada nodo que no es hoja en el árbol B tiene la forma

$$(p_0, k_1, p_1, k_2, p_2, \dots, k_n, p_n)$$

donde p_i es un apuntador al i -ésimo hijo del nodo, $0 \leq i \leq n$, y k_i es una clave, $1 \leq i \leq n$. Las claves dentro de un nodo están clasificadas en orden tal que $k_1 < k_2 < \dots < k_n$. Todas las claves del subárbol al que apunta p_0 son menores que k_1 . Para $1 \leq i < n$, todas las claves del subárbol apuntado por p_i tienen valores mayores o iguales que k_i y menores que k_{i+1} . Todas las claves en el subárbol apuntado por p_n son mayores que k_n .

Hay varias formas de organizar las hojas. Aquí se supondrá que el archivo principal está almacenado sólo en las hojas, y que cada hoja ocupa un bloque.

Recuperación

Para recuperar un registro r con clave x , se sigue el camino desde la raíz hasta la hoja que contiene a r , si es que existe en el archivo. Se sigue este camino pasando sucesivamente nodos interiores ($p_0, k_1, p_1, \dots, k_n, p_n$) del almacenamiento externo a la memoria principal y buscando la posición de x relativa a las claves $k_1, k_2, \dots, k_n$. Si $k_i \leq x < k_{i+1}$, a continuación se busca el nodo apuntado por p_i y se repite el proceso. Si $x < k_1$, se aplica p_0 para alcanzar el siguiente nodo; si $x \geq k_n$, se utiliza p_n . Cuando este proceso llega a una hoja, debe buscarse el registro con la clave x . Si el número de elementos de un nodo es pequeño, es posible utilizar la búsqueda lineal dentro del nodo, de otra forma, conviene utilizar una búsqueda binaria.

Inserción

La inserción en un árbol B es similar a la inserción en árboles 2-3. Para insertar un registro r con clave x en un árbol B , primero se aplica el procedimiento de búsqueda para localizar la hoja L a la cual corresponde r . Si existe espacio suficiente para r en L , se inserta r en el orden correspondiente. En este caso no se requieren modificaciones a los antecesores de L .

Si no hay espacio para r en L , es necesario pedir al sistema de archivos un bloque nuevo L' para pasar la segunda mitad de los registros de L a L' , insertando r en el lugar adecuado en L o L' . Sea el nodo P el padre de L ; P es conocido, ya que el procedimiento de búsqueda siguió un camino desde la raíz hasta L , pasando por P . Se aplica ahora el procedimiento de inserción recursivamente para colocar en P una clave k' y un apuntador l' a L' ; k' y l' se insertan inmediatamente después de la clave y del apuntador de L . El valor de k' es el de la clave más pequeña de L' .

Si P ya tiene m apuntadores, la inserción de k' y l' dentro de P hará que P se divida y requiera la inserción de una clave y un apuntador en el padre de P . Los efectos de esta inserción pueden transmitirse a los antecesores del nodo L en dirección a la raíz, por el camino que se siguió con el procedimiento de búsqueda original. Incluso puede ser necesario dividir la raíz, en cuyo caso se creará una nueva raíz con las dos mitades de la raíz anterior como sus dos hijos. Esta es la única situación en la que un nodo puede tener menos de $m/2$ hijos.

Eliminación

Para eliminar un registro r con una clave x , primero se encuentra la hoja L que contiene a r . Después, se elimina r de L , si existe. Si r es el primer registro en L , es necesario ir a P , el padre de L , para ajustar el valor de la clave en la entrada de P para que L sea el nuevo valor de la primera clave de L . Sin embargo, si L es el primer hijo de P , la primera clave de L no se registra en P , sino que aparece en alguno de

† Esta estrategia es la más sencilla de las varias respuestas aplicables a la situación en que ha de dividirse un bloque. En los ejercicios se mencionan algunas otras opciones que proporcionan una mayor ocupación media de los bloques con una inserción más laboriosa.

los antecesores de P , de forma específica, en el menor antecesor A tal que L no sea el descendiente más a la izquierda de A . Así pues, se debe propagar el cambio en la menor clave de L de vuelta por la trayectoria de la raíz a L .

Si L queda vacío después de la eliminación, se devuelve al sistema de archivos †; después se ajustan las claves y los apuntadores de P para reflejar la eliminación de L . Si el número de hijos de P ahora es menor que $m/2$, se examina el nodo P' situado inmediatamente a la izquierda (o a la derecha) de P en el mismo nivel del árbol. Si P' tiene por lo menos $\lceil m/2 \rceil + 1$ hijos, se distribuyen las claves y los apuntadores de P y P' en forma equitativa entre ambos, cuidando, por supuesto, el orden de clasificación, de manera que ambos nodos tengan por lo menos $\lceil m/2 \rceil$ hijos. Después, se modifican los valores de las claves para P y P' en el padre de P y, si fuera necesario, se propagan recursivamente los efectos de este cambio a todos los antecesores de P que se vean afectados.

Si P' tiene exactamente $\lceil m/2 \rceil$ hijos, es bueno combinar P y P' en un solo nodo con $2 \lceil m/2 \rceil - 1$ hijos (esto es, m hijos a lo sumo). Después, se deben quitar en el padre la clave y el apuntador correspondientes a P' . Esta eliminación puede hacerse con una aplicación recursiva del procedimiento de eliminación.

Si los efectos de la eliminación se propagan hasta la raíz, puede requerirse la combinación de los dos únicos hijos de ésta. En ese caso, el nodo combinado resultante quedará como nueva raíz, y la raíz anterior puede devolverse al sistema de archivos. La altura del árbol B se verá reducida en uno.

Ejemplo 11.5. Considérese el árbol B de orden 5 de la figura 11.10. Insertar el registro con clave 23 en este árbol produce el árbol B de la figura 11.11. Para insertar 23, se debe partir el bloque que contiene 22, 23, 24 y 26, ya que se ha supuesto que un bloque se llena con tres registros. Los dos registros más pequeños permanecen en el mismo bloque y los otros dos se colocan en un bloque nuevo. Un par apuntador-clave del nodo nuevo debe insertarse en el padre, que entonces se divide, porque no puede contener seis apuntadores. La raíz recibe el par apuntador-clave para el nodo nuevo, pero no se divide, porque tiene capacidad en exceso.

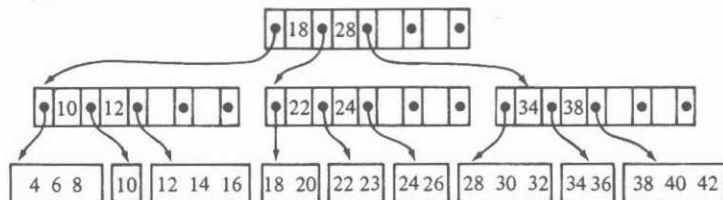


Fig. 11.11. Árbol B después de la inserción.

† Se pueden utilizar estrategias para evitar que los bloques hoja queden totalmente vacíos. Como ejemplo, a continuación se describe un esquema que impide que los nodos interiores queden ocupados en menos de la mitad de su capacidad, y esta técnica se puede aplicar también a las hojas, con un valor de m igual al mayor número de registros que caben en un bloque.

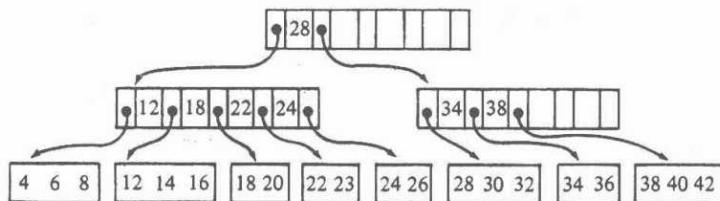


Fig. 11.12. Árbol B después de la eliminación.

La eliminación del registro 10 del árbol B de la figura 11.11 produce el árbol B de la figura 11.12. Aquí, el bloque que contiene a 10 se descarta; su padre queda con sólo dos hijos, y el hermano derecho tiene el número mínimo, tres; así, se combina el padre con su hermano, haciendo un nodo con cinco hijos. □

Análisis del tiempo de las operaciones con árboles B

Supóngase que se tiene un archivo con n registros organizado en un árbol B de orden m . Si cada hoja contiene en promedio b registros, el árbol tiene cerca de $[n/b]$ hojas. Los caminos más largos posibles se producen si cada nodo interior tiene la menor cantidad posible de hijos, esto es, $m/2$. En este caso, habrá unos $2 [n/b]/m$ padres de hojas, $4 [n/b]/m^2$ padres de padres de hojas, y así sucesivamente.

Si hay j nodos en el camino que va de la raíz a una hoja, entonces $2^{j-1} [n/b]/m^{j-1} \geq 1$, pues si no habría menos de un nodo en el nivel de la raíz. Por tanto, $[n/b] \geq (m/2)^{j-1}$, y $j \leq 1 + \log_{m/2} [n/b]$. Por ejemplo, si $n = 10^6$, $b = 10$ y $m = 100$, entonces $j \leq 3.5$. Obsérvese que b no es el número máximo de registros que se pueden poner en un bloque, sino un promedio, o número esperado. Sin embargo, redistribuyendo los registros entre bloques vecinos cuando uno se vacía hasta menos de la mitad, se puede asegurar que b es por lo menos la mitad del valor máximo. Obsérvese también que se ha supuesto que cada nodo interior tiene el mínimo número posible de hijos; en la práctica, el nodo interior promedio tendrá más que el mínimo, por lo que el análisis anterior resulta conservador.

Para una inserción o eliminación, se necesitan j accesos a bloques para localizar la hoja apropiada. El número exacto de accesos adicionales a bloques necesario para culminar la inserción o la eliminación, y distribuir sus efectos por el árbol, es difícil de calcular. La mayor parte de las veces sólo es necesario escribir de nuevo un bloque, la hoja que contiene al bloque de interés. Así, $2 + \log_{m/2} [n/b]$ puede tomarse como el número aproximado de accesos a bloques para inserción o eliminación.

Comparación de métodos

Se ha estudiado la función de dispersión, los índices dispersos y los árboles B como métodos posibles de organización de archivos externos. Es interesante comparar en

cada método el número de accesos a bloques relacionados con una operación con archivos.

La función de dispersión suele ser el método más rápido de los tres, y requiere en promedio dos accesos a bloques por cada operación (excluyendo los accesos requeridos para buscar la tabla de cubetas), si el número de cubetas es lo bastante grande para que la cubeta típica use sólo un bloque. Sin embargo, con la función de dispersión no es fácil acceder a los registros en el orden de clasificación.

Un índice disperso en un archivo de n registros permite que las operaciones sobre archivos se realicen en unos $2 + \log(n/bb')$ accesos a bloques mediante la búsqueda binaria; aquí, b es el número de registros que caben en un bloque y b' es el número de pares apuntador-clave que caben en un bloque del archivo índice. Los árboles B permiten operaciones con archivos en $2 + \log_{m/2} [n/b]$ accesos a bloques, donde m , el grado máximo de los nodos interiores, es aproximadamente b' . Los índices dispersos y los árboles B permiten el acceso a los registros en el orden de clasificación.

Todos estos métodos son muy buenos comparados con la búsqueda secuencial obvia en un archivo. Las diferencias de tiempo entre ellos, no obstante, son pequeñas y difíciles de determinar analíticamente, especialmente considerando que los parámetros importantes, como la longitud esperada del archivo y la razón de ocupación de bloques, son difíciles de predecir.

Parece ser que los árboles B se están popularizando con rapidez como medio de acceso a archivos en sistemas de bases de datos. En parte, se debe a su capacidad para manejar consultas de registros con claves en un intervalo determinado (lo que aprovecha el hecho de que los registros aparecen ordenados en el archivo principal). El índice disperso también maneja con eficiencia dichas consultas, pero es casi seguro que es menos eficiente que los árboles B. Intuitivamente, la razón de que los árboles B sean superiores a los índices dispersos es que se puede considerar un árbol B como un índice disperso sobre un índice disperso sobre un índice disperso, y así sucesivamente (aunque rara vez se necesitan más de tres niveles de índices).

Los árboles B también funcionan relativamente bien cuando se usan como índices secundarios, donde las «claves» no definen unívocamente un registro. Aunque los registros con un valor dado en los campos designados de un índice secundario se extienden sobre muchos bloques, se pueden leer todos con un número de accesos a bloque igual al número de bloques que contienen esos registros, más el número de sus antecesores en el árbol B. En comparación, si esos registros, más otro grupo de tamaño similar tienen la misma función de dispersión y llegan a la misma cubeta, entonces la recuperación de cualquier grupo de una tabla de dispersión puede requerir un número de accesos a bloques cercano al doble del número de bloques en los cuales puede caber cada grupo. Es factible que haya otras razones en favor de los árboles B, como su rendimiento cuando varios procesos tienen acceso simultáneamente a la estructura, pero eso está fuera del alcance de este libro.

Ejercicios

- 11.1 Escribase un programa *concatena* que tome una secuencia de nombres de archivos como argumentos y escriba el contenido de los mismos en la salida estándar, concatenando así los archivos.
- 11.2 Escribase un programa *incluye* que copie su entrada en su salida, excepto cuando encuentre una línea de la forma *#incluye archivo*, en cuyo caso reemplaza esta línea por el contenido del archivo mencionado. Obsérvese que los archivos incluidos también pueden contener proposiciones *#incluye*.
- 11.3 ¿Cómo se comporta el programa del ejercicio 11.2 cuando un archivo se incluye a sí mismo?
- 11.4 Escribase un programa *compara* que compare dos archivos, registro por registro, para determinar si son iguales.
- *11.5 Reescribase el programa de comparación de archivos del ejercicio 11.4 con el algoritmo SCL de la sección 5.6 para encontrar la subsecuencia común más larga de registros en ambos archivos.
- 11.6 Escribase un programa *encuentra* que tome dos argumentos que consten de una cadena y un nombre de archivo, e imprima todas las líneas del archivo que contengan la cadena como una subcadena. Por ejemplo, si la cadena es «ado» y el archivo es una lista de palabras, entonces *encuentra* imprime todas las palabras que contienen el trigramo «ado».
- 11.7 Escribase un programa que lea un archivo y escriba en su salida estándar los registros del archivo clasificados.
- 11.8 ¿Cuáles son las primitivas que Pascal ofrece para tratar con archivos externos? ¿Cómo pueden mejorarse?
- *11.9 Supóngase que se maneja una clasificación en varias fases con tres archivos, y en la i -ésima fase se crea un archivo con r_i fragmentos de longitud l_i . En la n -ésima fase, se desea un fragmento en uno de los archivos y ninguno en los otros dos. Explíquese por qué cada uno de los siguientes enunciados debe ser cierto.
- $l_i = l_{i-1} + l_{i-2}$ para $i \geq 1$, donde l_0 y l_{-1} se consideran las longitudes de los fragmentos en los dos archivos ocupados inicialmente.
 - $r_i = r_{i-2} - r_{i-1}$ (o, en forma equivalente, $r_{i-2} = r_{i-1} + r_i$ para $i \geq 1$), donde r_0 y r_{-1} son el número de fragmentos en los dos archivos iniciales.
 - $r_n = r_{n-1} = 1$ y, por tanto, $r_n, r_{n-1}, \dots, r_1$, forma una sucesión de Fibonacci.
- *11.10 ¿Qué condición debe agregarse a las del ejercicio 11.9 para hacer posible una clasificación en varias fases,
- con fragmentos iniciales de longitud uno (esto es, $l_0 = l_{-1} = 1$)?
 - la ejecución para k fases, pero con fragmentos iniciales diferentes a uno permitido?

Sugerencia. Considérense unos cuantos ejemplos, como $l_n = 50$, $l_{n-1} = 31$ o $l_n = 50$, $l_{n-1} = 32$.

****11.11** Generalícense los ejercicios 11.9 y 11.10 a clasificaciones de varias fases con más de tres archivos.

****11.12** Muéstrese que:

- Cualquier algoritmo de clasificación externa que utilice sólo una cinta como almacenamiento externo debe requerir un tiempo $\Omega(n^2)$ para clasificar n registros.
- Un tiempo $O(n \log n)$ es suficiente si hay dos cintas para almacenamiento externo.

11.13 Supóngase que se tiene un archivo externo de arcos dirigidos $x \rightarrow y$ que forma un grafo acíclico, y que no hay suficiente espacio en la memoria interna para contener el conjunto completo de vértices o aristas al mismo tiempo.

- Escríbese un programa de clasificación topológica externa que escriba un ordenamiento lineal de vértices, de modo que si $x \rightarrow y$ es un arco dirigido, el vértice x aparezca antes de y en el ordenamiento lineal.
- ¿Cuál es la complejidad de tiempo y espacio de este programa como una función del número de accesos a bloques?
- ¿Qué haría este programa si el grafo dirigido fuera cíclico?
- **d)** ¿Cuál es el número mínimo de accesos a bloque necesarios para clasificar topológicamente un grafo dirigido acíclico almacenado externamente?

11.14 Supóngase que se tiene un archivo de un millón de registros, donde cada registro ocupa 100 bytes. Los bloques son de 1000 bytes de longitud, y un apuntador a un bloque ocupa 4 bytes. Diseñese una organización con función de dispersión para este archivo. ¿Cuántos bloques se necesitan para la tabla de cubetas y las cubetas?

11.15 Diseñese una organización con árboles B para el archivo del ejercicio 11.14.

11.16 Escribanse programas para implantar las operaciones RECUPERA, INSERTA, SUPRIME y MODIFICA en

- archivos con función de dispersión,
- archivos indizados,
- archivos con árboles B.

11.17 Escribese un programa para encontrar el k -ésimo elemento más grande en

- un archivo con índice disperso
- un archivo con árbol B

- *11.18 Supóngase que se necesitan $a + bm$ milisegundos para leer un bloque que contiene un nodo de un árbol m -ario de búsqueda, y $c + d \log_2 m$ milisegundos para procesar cada nodo en memoria interna. Si hay n nodos en el árbol, es necesario leer cerca de $\log_m n$ nodos para localizar un registro dado. Por tanto, el tiempo total requerido para encontrar un registro dado en el árbol es

$$(\log_m n)(a + bm + c + d \log_2 m) = (\log_2 n)((a + c + bm)/\log_2 m + d)$$

milisegundos. Háganse estimaciones razonables para los valores de a , b , c y d , y represéntese gráficamente esta cantidad como una función de m . ¿Para qué valor de m se obtiene el mínimo?

- *11.19 Un árbol B^* es un árbol B en el que cada nodo interno está lleno en dos terceras partes por lo menos (en vez de la mitad). Diseñese un esquema de inserción para árboles B^* que retrase la división de nodos internos hasta que dos nodos hermanos estén llenos. Los dos nodos hermanos pueden entonces dividirse en tres, cada uno lleno en dos terceras partes. ¿Qué ventajas y desventajas tienen los árboles B^* en relación con los árboles B ?
- *11.20 Cuando la clave de un registro es una cadena de caracteres, se puede ahorrar espacio almacenando sólo un prefijo de la clave como separador clave de cada nodo interno de un árbol B . Por ejemplo, «gato» y «perro», pueden separarse por el prefijo «g» o «ga» de «gato». Diseñese un algoritmo de inserción en árboles B que utilice prefijos de claves como separadores que sean siempre lo más cortos posible.
- *11.21 Supóngase que en cierto archivo las operaciones de inserción y eliminación se efectúan en una fracción p del tiempo, y en el tiempo $1-p$ restante se realizan recuperaciones donde se especifica exactamente un campo. Hay k campos en los registros, y una recuperación especifica el i -ésimo campo con probabilidad q_i . Supóngase también que una recuperación requiere a milisegundos si no hay índice secundario para el campo especificado, y b milisegundos, si lo hay, y que una inserción o eliminación requiere $c + sd$ milisegundos, donde s es el número de índices secundarios. Determínese, como una función de a , b , c , d , p y las q_i , qué índices secundarios deben crearse para el archivo, de manera que el tiempo promedio por operación se minimice.
- *11.22 Supóngase que las claves son de un tipo que puede ordenarse linealmente, como los números reales, y que se conoce la distribución de probabilidades con que aparecerán las claves de valores dados en el archivo. Se puede aprovechar este conocimiento para mejorar una búsqueda binaria cuando se busca una clave en un índice disperso. Un esquema, llamado *búsqueda por interpolación*, usa esta información estadística para predecir dónde es más probable que se encuentre una clave x , en el intervalo de bloques de índices $B_1, \dots, B_p$, a los cuales se ha limitado la búsqueda. Proporcionese

- a) un algoritmo para aprovechar el conocimiento estadístico en esta forma, y
- b) una demostración de que $O(\log \log n)$ accesos a bloques son suficientes, en promedio, para encontrar una clave.

11.23 Supóngase que se tiene un archivo externo de registros, y que cada registro consta de una arista de un grafo G y un costo asociado a esa arista.

- a) Escribese un programa para construir un árbol abarcador de costo mínimo para G , suponiendo que existe suficiente memoria para almacenar todos los vértices de G , pero no todas las aristas.
- b) ¿Cuál es la complejidad de tiempo de ese programa como una función del número de vértices y aristas?

Sugerencia. Un enfoque posible de este problema es mantener en memoria un bosque con los componentes conexos actuales. Cada arista es leída y procesada como sigue: si la siguiente arista tiene extremos en dos componentes distintos, se agrega y se mezclan los componentes. Si la arista crea un ciclo en un componente ya existente, se agrega y se elimina la arista de mayor costo del ciclo (que puede ser la arista actual). Este enfoque es similar al algoritmo de Kruskal, pero no requiere clasificar las aristas, lo cual es un aspecto importante de este problema.

11.24 Supóngase que se tiene un archivo que contiene una secuencia de números positivos y negativos $a_1, a_2, \dots, a_n$. Escribese un programa $O(n)$ para encontrar una subsecuencia contigua $a_i, a_{i+1}, \dots, a_j$ que tenga la suma mayor $a_i + a_{i+1} + \dots + a_j$ de cualquier subsecuencia.

Notas bibliográficas

Como material adicional sobre clasificación externa, véase Knuth [1973]. Material posterior sobre estructuras de datos externas y su uso en sistemas de bases de datos puede encontrarse en esa obra y en Ullman [1982] y Wiederhold [1982]. La clasificación en varias fases se estudia en Shell [1971]. El esquema de intercalación con seis buffers de la sección 11.2 es de Friend [1956], y el de cuatro, de Knuth [1973].

La selección de índices secundarios, de la cual el ejercicio 11.21 es una simplificación, se analiza en Lum y Ling [1970] y Schkolnick [1975]. Los árboles B se presentaron originalmente en Bayer y McCreight [1972]. En Comer [1979], se examinan muchas variantes, y en Gudes y Tsur [1980] se evalúa el rendimiento en la práctica.

La información acerca del ejercicio 11.12, clasificación con una y dos cintas, puede encontrarse en Floyd y Smith [1973]. El ejercicio 11.22 sobre búsqueda por in-

terpolación se analiza con detalle en Yao y Yao [1976], y Perl, Itai y Avni [1978].

Una implantación excelente del enfoque sugerido en el ejercicio 11.23 para el problema del árbol abarcador externo de costo mínimo fue estudiada por V. A. Vyssotsky, alrededor de 1960 (sin publicar). El ejercicio 11.24 se debe a M. I. Shamos.

12

Administración de memoria

En este capítulo se analizan las estrategias básicas para reutilizar el espacio en memoria o compartirlo entre objetos distintos que crecen y se contraen de manera arbitraria. Por ejemplo, se estudiarán los métodos que mantienen listas enlazadas de espacio disponible, y técnicas de «recolección de basura», que sirven para conocer la disponibilidad de espacio sólo cuando parece que se ha terminado el espacio disponible.

12.1 Aspectos de la administración de memoria

En la operación de sistemas de cómputo existen muchas situaciones en las que se *administra* un recurso limitado de memoria, es decir, se comparte entre varios «compitidores». Un programador que no se ocupe de la realización de programas del sistema (compiladores, sistemas operativos, etcétera) tal vez no perciba dichas actividades, debido a que suelen realizarse «entre bastidores». Como ejemplo, los programadores de Pascal saben que el procedimiento *new(p)* hará que el apuntador *p* señale hacia un objeto nuevo del tipo correcto; pero, ¿de dónde procede el espacio para el objeto? El procedimiento *new* tiene acceso a una región grande de memoria, conocida como «estructura dinámica» (*heap*), que las variables del programa no usan. De esa región, se selecciona un bloque no utilizado de bytes consecutivos suficiente para contener un objeto del tipo al que apunta *p*, y se hace que *p* contenga la dirección del primer byte de ese bloque. Pero ¿cómo sabe el procedimiento *new* qué bytes de la memoria están «desocupados»? La sección 12.4 sugiere la respuesta.

Aún más misterioso es lo que sucede si se modifica el valor de *p*, bien por una asignación o por otra llamada a *new(p)*. El bloque de memoria al que apunta *p* puede ser ahora *inaccesible*, en el sentido de que no hay forma de llegar a él mediante las estructuras de datos del programa, y se puede reutilizar su espacio. Por otro lado, antes de cambiar *p*, su valor pudo haberse copiado en otra variable. En ese caso, el bloque de memoria será parte todavía de las estructuras de datos del programa. ¿Cómo se puede saber si un bloque de la región de memoria utilizada por el procedimiento *new* ya no es requerido por el programa?

El tipo de administración de memoria que se efectúa en Pascal sólo es uno más. Por ejemplo, en algunas situaciones, como Pascal, objetos de tamaños distintos comparten el mismo espacio de memoria; en otras situaciones, todos los objetos que comparten el espacio son del mismo tamaño. Esta distinción con respecto a los tamaños

de los objetos es una forma de clasificar las clases de problemas de administración de memoria a que uno debe enfrentarse. A continuación se presentan algunos ejemplos más.

1. En el lenguaje de programación LISP, el espacio de memoria se divide en *celdas* que, en esencia, son registros que constan de dos campos; cada campo puede contener un *átomo* (un objeto del tipo elemental, como puede ser un entero) o un apuntador a una celda. Los átomos y apuntadores son del mismo tamaño, así que todas las celdas requieren el mismo número de bytes. Todas las estructuras de datos conocidas pueden construirse a partir de esas celdas. Por ejemplo, las listas enlazadas de átomos pueden usar los primeros campos de las celdas para contener átomos, y los segundos campos, para contener apuntadores a las siguientes celdas de la lista. Los árboles binarios pueden representarse utilizando el primer campo de cada celda para apuntar al hijo izquierdo, y el segundo, para apuntar al hijo derecho. Al ejecutar un programa en LISP, el espacio de memoria empleado para contener una celda puede ser a la vez parte de estructuras distintas en momentos diferentes, ya sea porque una celda se mueve entre estructuras, o porque se separa de todas las estructuras y su espacio se utiliza de nuevo.
2. Un sistema de archivos, en general, divide los dispositivos de almacenamiento secundario, como los discos, en bloques de longitud fija. Por ejemplo, UNIX siempre usa bloques de 512 bytes. Los archivos se almacenan en una secuencia de bloques (que no son necesariamente consecutivos). Conforme se crean y destruyen archivos, los bloques del almacenamiento secundario quedan disponibles para ser utilizados de nuevo.
3. Un sistema operativo de multiprogramación típico permite que varios programas compartan la memoria principal al mismo tiempo. Cada programa requiere una cantidad determinada de memoria, la cual es conocida por el sistema operativo, y este requisito es parte de la solicitud de servicio emitida cuando se desea ejecutar el programa. Mientras en los ejemplos (1) y (2) los objetos que comparten memoria (celdas y bloques, respectivamente) eran todos del mismo tamaño, distintos programas requieren cantidades diferentes de memoria. Así, cuando termina un programa que usa 100K bytes, puede reemplazarse por otros dos que manejen 50K cada uno, o uno 20K y otro 70K (con 10K no utilizados). Como solución distinta, los 100K bytes liberados a la terminación del programa pueden combinarse con los 50K adyacentes que estén sin utilizar, y entonces puede ejecutarse un programa que requiera hasta 150K. Otra posibilidad es que ningún programa nuevo quepa en el espacio liberado, y que 100K bytes quede momentáneamente sin utilizar.
4. Hay muchos lenguajes de programación, como SNOBOL, APL o SETL, que asignan espacio a objetos de tamaño arbitrario. A esos objetos, que son valores asignados a variables, se les asigna un bloque de espacio de un gran bloque de memoria, que suele denominarse *estructura dinámica* (*heap*). Cuando cambia al valor de una variable, al valor nuevo se le asigna un espacio en la estructura dinámica, y un apuntador para que la variable apunte al nuevo valor. Es posible que el valor anterior de la variable quede ahora sin utilizar, y su espacio se pueda volver a utilizar. Sin embargo, los lenguajes como SNOBOL o SETL realizan asig-

naciones como $A = B$ haciendo que el apuntador de A apunte al mismo objeto que el apuntador de B ; si A o B fueran reasignados, el objeto anterior no se liberaría y su espacio no podría solicitarse.

Ejemplo 12.1. En la figura 12.1(a) se observa la estructura dinámica que puede manejarse en un programa en SNOBOL con las variables A , B y C . El valor de cualquier variable en SNOBOL es una cadena de caracteres y, en este caso, el valor de A y B es «BUENOS DIAS» y el valor de C es «PASA LA SAL».

Se ha elegido la representación de cadenas de caracteres por medio de apuntadores a bloques de memoria en la estructura dinámica. Esos bloques tienen sus dos primeros bytes (el número 2 es un valor típico que podría cambiarse) dedicados a un entero que da la longitud de la cadena; por ejemplo, «BUENOS DIAS» tiene longitud 11, contando el espacio entre palabras, así que el valor de A (y de B) ocupa 13 bytes.

Si el valor de B se cambia por «BUENAS NOCHES», se puede encontrar un bloque vacío en el montón con 15 bytes para almacenar el nuevo valor de B , incluyendo los dos bytes de la longitud. Se hace que el apuntador de B apunte al nuevo valor, como se muestra en la figura 12.1(b). El bloque que contiene el entero 11 y «BUENOS DIAS» aún es útil, pues A continúa apuntando a él. Si el valor de A cambia, ese bloque quedará sin uso y puede volver a utilizarse. La forma de saber que no hay apuntadores a tales bloques es un tema importante en este capítulo □

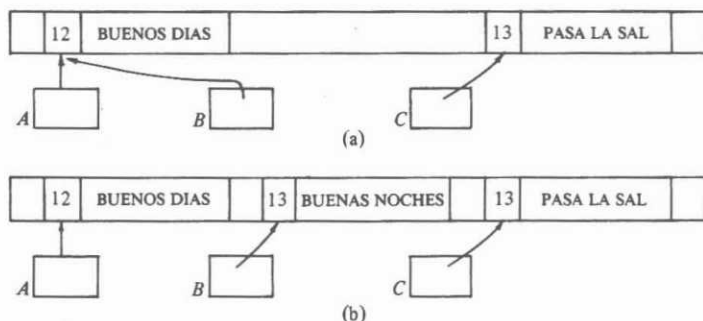


Fig. 12.1. Variables de cadenas en una estructura dinámica.

En los cuatro ejemplos anteriores, se pueden observar diferencias a lo largo, por lo menos, de dos «dimensiones» ortogonales. El primer aspecto es si los objetos que se encuentran compartiendo el espacio son de la misma longitud o no. En los dos primeros ejemplos, los programas en LISP y el almacenamiento de archivos, los objetos, celdas tipo LISP en un caso, y bloques con partes de archivos, en el otro, son del mismo tamaño. Este hecho permite ciertas simplificaciones del problema de la administración de memoria. Por ejemplo, en la realización en LISP, una región de memoria se divide en espacios, cada uno de los cuales puede contener exactamente una celda. El problema de la administración es encontrar espacios vacíos para ubi-

car las celdas recién creadas y nunca se necesita almacenar una celda en una posición en que se traslapen dos espacios. Igualmente, en el segundo ejemplo, un disco se divide en bloques de tamaño idéntico, y a cada bloque se le asigna parte de un archivo; nunca se emplea un bloque para almacenar partes de dos o más archivos, aunque un archivo termine en mitad de un bloque.

En contraste, el tercero y cuarto ejemplos cubren la asignación de memoria para un sistema de multiprogramación y administración de una estructura dinámica para los lenguajes que tratan con variables cuyos valores son objetos «grandes», y aquí se habla de asignación de espacio en bloques de tamaño diferente. Este requisito presenta ciertos problemas que no se presentan en el caso de longitudes fijas. Por ejemplo, se teme la *fragmentación*, una situación en la cual hay mucho espacio sin utilizar, pero está distribuido en fragmentos tan pequeños que no puede encontrarse espacio para un objeto grande. Se profundizará más acerca de la administración de estructuras dinámicas en las secciones 12.4 y 12.5.

El segundo aspecto importante es si la *recolección de basura*, un término muy descriptivo para la recuperación de espacio no utilizado, se efectúa explícita o implícitamente, esto es, mediante un mandato del programa o sólo como respuesta a una petición de espacio que no puede satisfacerse de otra manera. En el caso de la administración de archivos, cuando se elimina un archivo, los bloques que se utilizaron para contenerlo son conocidos por el sistema de archivos. Por ejemplo, el sistema de archivos puede grabar la dirección de uno o más «bloques maestros» para cada archivo existente; los bloques maestros listan las direcciones de todos los bloques utilizados por el archivo. Así, cuando se elimina un archivo, el sistema de archivos puede hacerlo explícitamente disponible para reutilizar todos los bloques empleados por ese archivo.

En contraste, las celdas de LISP continúan ocupando su espacio en memoria cuando se apartan de las estructuras de datos del programa. Debido a la posibilidad de que haya varios apuntadores a una celda, no se puede decir cuándo una celda está separada por completo y, por tanto, tampoco recoger en forma explícita las celdas, como se hizo con los bloques de un archivo eliminado. Tarde o temprano, todos los espacios en memoria corresponderán a celdas útiles o inútiles, y la siguiente solicitud de espacio para otra celda activará implícitamente una «recolección de basura»; en ese momento, el intérprete de LISP marca todas las celdas útiles, por medio de un algoritmo similar al que se presentará en la sección 12.3, y después enlazará todos los bloques que contengan celdas inútiles en una lista de espacio disponible, para poderlas reutilizar.

La figura 12.2 ilustra las cuatro clases de administración de memoria y da un ejemplo de cada una. De la figura 12.2 ya se han comentado los ejemplos de bloques de tamaño fijo. La administración de la memoria principal en un sistema de programación múltiple es un ejemplo de petición explícita de bloques con longitud variable. Esto es, cuando un programa termina, el sistema operativo, sabiendo qué área de memoria se otorgó al programa y sabiendo que ningún otro programa puede usar ese espacio, hace que el espacio quede inmediatamente disponible para cualquier otro programa.

La administración de una estructura dinámica en SNOBOL o en muchos otros lenguajes es un ejemplo de bloques con longitud variable y recolección de basura.

		recuperación del espacio no utilizado	
		explícita	recolección de basura
tamaño del bloque	fijo	sistema de archivos	LISP
	variable	sistema de multiprogramación	SNOBOL

Fig. 12.2. Ejemplos de las cuatro estrategias de administración de memoria.

Como en LISP, un intérprete típico de SNOBOL no intenta recuperar bloques de memoria hasta que se agota el espacio. En ese momento, el intérprete realiza una recolección de basura igual que lo hace un intérprete de LIPS, pero con la posibilidad adicional de que las cadenas se muevan en torno a la estructura dinámica para reducir la fragmentación, y que los bloques libres adyacentes se combinen para formar bloques más grandes. Obsérvese que los dos últimos pasos no tienen sentido en un ambiente LISP.

12.2 Administración de bloques de igual tamaño

Supóngase que se tiene un programa que maneja celdas con un par de campos cada una; cada campo puede ser un apuntador a una celda o puede contener un «átomo». Por supuesto, la situación es igual a la de un programa escrito en LISP, pero el programa puede escribirse casi en cualquier lenguaje, incluso en Pascal, si se definen las celdas del tipo registro variante. Las celdas vacías que se encuentran disponibles para su incorporación a una estructura de datos se colocan en una lista de espacio disponible, y cada variable del programa se representa por un apuntador a una celda; la celda apuntada puede pertenecer a una gran estructura de datos.

Ejemplo 12.2. En la figura 12.3, se observa una estructura de datos posible. *A*, *B* y *C* son variables, y las letras minúsculas representan átomos. Obsérvense algunos fenómenos interesantes: la celda que contiene el átomo *a* está apuntada por la variable *A* y por otra celda; la celda que contiene el átomo *c* está apuntada por dos celdas distintas; las celdas que contienen *q* y *h* son un caso especial, porque aunque se apuntan mutuamente, no son accesibles desde las variables *A*, *B* o *C*, ni están en la lista de espacio disponible. □

Supóngase que cuando se ejecuta el programa, se pueden quitar celdas nuevas de la lista de espacio disponible; por ejemplo, puede ser conveniente sustituir el apuntador nulo de la celda con el átomo *c* de la figura 12.3 por un apuntador a una celda nueva que contenga el átomo *i* y un apuntador nulo. Esta celda se eliminará de la

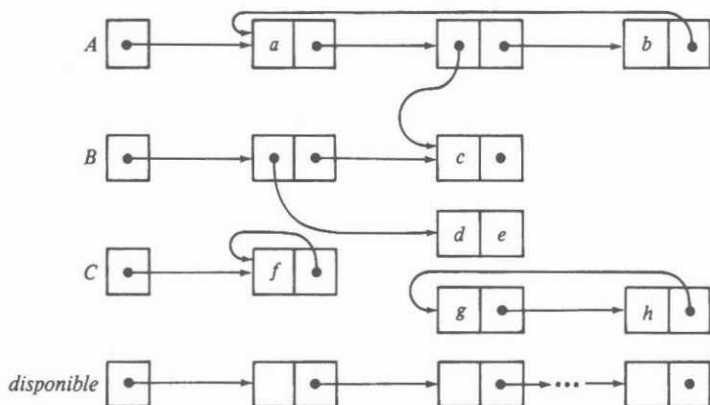


Fig. 12.3. Red de celdas.

cabeza de la lista de espacio disponible. También es posible que de vez en cuando los apuntadores cambien, de forma que las celdas se separen de las variables del programa, como sucedió con las celdas *g* y *h* de la figura 12.3. Por ejemplo, la celda en que se encuentra el átomo *c*, pudo haber apuntado alguna vez a la celda que contiene a *g*. Como otro ejemplo, el valor de la variable *B* puede cambiar en un momento dado, lo cual si nada más ha cambiado, aparta la celda apuntada por *B* en la figura 12.3, al igual que la celda que contiene a *d* y *e* (pero no la celda que contiene a *c*, pues ésta es accesible desde *A*). Las celdas que no pueden alcanzarse desde ninguna variable y que no están en la lista de espacio disponible son conocidas como celdas *inaccesibles*.

Cuando se separan las celdas, y no las vuelve a necesitar el programa, sería deseable que volvieran a la lista de espacio disponible, para poderlas usar de nuevo. Si no se reclaman dichas celdas, con el tiempo se llegará a la situación inaceptable de que el programa no esté empleando todas las celdas, aunque requiera una celda nueva, y la lista de espacio disponible esté vacía. Es entonces cuando debe realizarse una recolección de basura, lo que consume tiempo. Este paso de recolección de basura es «implícito», en el sentido de que no lo llamó explícitamente la petición de espacio.

Cuentas de referencia

Un enfoque atractivo para la detección de celdas inaccesibles consiste en incluir en cada celda una *cuenta de referencia*, esto es, un campo de tipo entero cuyo valor sea igual al número de apuntadores a la celda. Es fácil mantener las cuentas de referencia; cuando se hace que un apuntador apunte a una celda, se agrega un uno a la cuenta de referencia de esa celda, y cuando se reasigna un apuntador no nulo, primero

disminuye en uno la cuenta de referencia de la celda apuntada. Si una cuenta de referencia llega a valer cero, la celda será inaccesible y se podrá devolver a la lista de disponibles.

Lamentablemente, las cuentas de referencia no siempre funcionan. Las celdas con *g* y *h* de la figura 12.3 son inaccesibles y enlazadas en un ciclo. Sus cuentas de referencia valen 1, de manera que no se pueden devolver a la lista de disponibles. Se puede intentar detectar ciclos de celdas inaccesibles de distintas formas, pero tal vez no valga la pena hacerlo. Las cuentas de referencia son útiles para estructuras que no tienen ciclos de apuntadores. Un ejemplo de estructura sin posibilidades de ciclos es una colección de variables apuntando a bloques que contienen datos, como en la figura 12.1. De esta forma, se puede hacer la recolección de basura de forma explícita, recogiendo un bloque cuando su cuenta de referencia ha alcanzado el valor cero. Sin embargo, cuando las estructuras de datos permiten ciclos de apuntadores, la estrategia basada en cuentas de referencia por lo general es inferior a otro enfoque que se analizará en la siguiente sección, tanto en función del espacio requerido por las celdas como del tiempo relacionado con el caso de las celdas inaccesibles.

12.3 Algoritmos de recolección de basura para bloques de igual tamaño

Ahora se presenta un algoritmo para detectar qué celdas de una colección de los tipos sugeridos en la figura 12.3 son accesibles desde las variables del programa. Se definirá con precisión el problema definiendo un tipo de celda en Pascal que es una variante de tipo registro; las cuatro variantes, que se llamarán *PP*, *PA*, *AP*, *AA*, se determinan según cuáles de los dos campos de datos son apuntadores y cuáles son átomos. Por ejemplo, *PA* significa que el campo izquierdo es un apuntador y el campo derecho es un átomo. Un campo booleano adicional en las celdas, llamado *marca*, indica si la celda es accesible. Es decir, poniendo *marca* en verdadero al hacer la recolección de basura, se «marca» la celda, indicando que es accesible. En la figura 12.4 se muestran las definiciones de los tipos.

type

```

tipo_átomo = { algún tipo apropiado, de preferencia,
                del mismo tamaño que los apuntadores }
patrones = (PP, PA, AP, AA);
tipo_celda = record
    marca: boolean;
    case patrón: patrones of
        PP: (izquierda: ↑ tipo_celda; derecha: ↑ tipo_celda);
        PA: (izquierda: ↑ tipo_celda; derecha: tipo_átomo);
        AP: (izquierda: tipo_átomo; derecha: ↑ tipo_celda);
        AA: (izquierda: tipo_átomo; derecha: tipo_átomo);
    end;
end;
```

Fig. 12.4. Definición del tipo de las celdas.

Se supone que hay un arreglo de celdas, que ocupa casi toda la memoria, y una colección de variables, que son apuntadores a las celdas. Por simplicidad, se da por hecho que hay sólo una variable, llamada *fuelle*, apuntando a una celda, pero la extensión a muchas variables no es difícil †. Esto es, se declara

```
var
    fuente: † tipo_celda;
    memoria: array [1..tamaño_memoria] of tipo_celda;
```

Para marcar las celdas accesibles desde *fuelle*, primero se «desmarcan» todas las celdas, sean o no accesibles, recorriendo todo el arreglo *memoria* y poniendo el campo *marca* en falso. Después, se realiza una búsqueda primera de profundidad en el grafo que emana de *fuelle*, marcando todas las celdas visitadas. Las celdas visitadas son exactamente aquellas que son accesibles. Después, se recorre el arreglo *memoria* para agregar a la lista de espacio disponible todas las celdas no marcadas. La figura 12.5 muestra un procedimiento *bpf* para realizar la búsqueda en profundidad; *bpf* se llama con el procedimiento *recoge* que desmarca todas las celdas, y marca las celdas accesibles llamando a *bpf*. No se presenta el código para enlazar la lista de espacio disponible debido a las peculiaridades de Pascal. Por ejemplo, aunque se pueden enlazar las celdas disponibles con todas las celdas izquierdas o todas las derechas, ya que se ha supuesto que los apuntadores y los átomos tienen el mismo tamaño, no está permitido reemplazar átomos por apuntadores en las celdas que sean del tipo variante *AA*.

```
(1)  procedure bpf ( celda_actual: † tipo_celda );
      { Si se marcó la celda actual no se hace nada; en caso contrario,
        debe marcarse y llamar a bpf en cualquier celda apuntada por
        la celda actual }
      begin
(2)      with celda_actual † do
(3)          if marca = false then begin
(4)              marca := true;
(5)              if (patrón = PP) or (patrón = PA) then
(6)                  if izquierda <> nil then
(7)                      bpf(izquierda);
(8)              if (patrón = PP) or (patrón = AP) then
(9)                  if derecha <> nil then
(10)                     bpf(derecha);
          end
      end; { bpf }

(11) procedure recoge;
      var
          i: = integer;
```

† Cada lenguaje de programación debe proporcionar un método propio de representación del conjunto de variables actual, y cualquiera de los métodos estudiados en los capítulos 4 y 5 es adecuado. Por ejemplo, la mayor parte de las aplicaciones usan una tabla de dispersión para guardar las variables.

```

begin
(12)   for  $i := 1$  to tamaño_memoria do { "desmarcar" todas las celdas }
(13)       memoria[i].marca := false;
(14)       bpf(fuente); { marcar las celdas accesibles }
(15)       { aquí va el código para la recogida }
end; { recoge }

```

Fig. 12.5. Algoritmo para marcar celdas accesibles.

Recolección en el mismo sitio

El algoritmo de la figura 12.5 tiene un defecto sutil; en un ambiente de programación donde la memoria es limitada, puede suceder que no haya espacio disponible para almacenar la pila requerida para las llamadas recursivas a *bpf*. Como se ilustró en la sección 2.6, cada vez que *bpf* se llama a sí mismo, Pascal (o cualquier otro lenguaje que permita la recursión) crea un «registro de activación» para esa llamada particular a *bpf*. En general, un registro de activación contiene espacio para parámetros y variables locales al procedimiento, del cual cada llamada necesita su propia copia. Algo que también es necesario en cada registro de activación es una «dirección de retorno», el lugar al cual debe regresar el control cuando termina esta llamada recursiva al procedimiento.

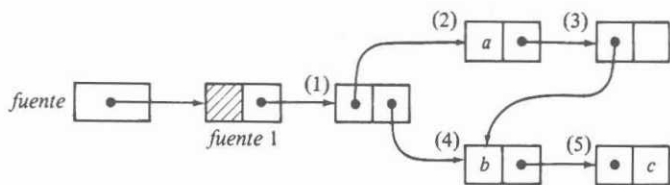
En el caso particular de *bpf*, sólo se necesita espacio para el parámetro y la dirección de retorno [esto es, fue llamado desde la línea (14) de *recoge*, desde la línea (7) o la (10) de otra invocación de *bpf*]. No obstante, esta solicitud de espacio es suficiente para que, si toda la memoria se enlaza en una sola cadena extendida desde *fuelle* (por lo que el número de llamadas activas a *bpf* puede ser en algún momento igual a la longitud de esta cadena), se requerirá bastante más espacio para la pila de registros de activación que el asignado para la memoria. Si ese espacio quizá no estuviera disponible, sería imposible realizar la marcación.

Por fortuna, hay un ingenioso algoritmo, conocido como *algoritmo de Deutsch-Schorr-Waite*, para hacer la marcación en el mismo sitio. Es necesario convencerse de que la secuencia de celdas sobre la cual se ha realizado una llamada a *bpf*, sin haber terminado, en realidad forma un camino desde *fuelle* hasta la celda en la cual se hizo la llamada actual a *bpf*. Así, se puede usar una versión no recursiva de *bpf*, y en vez de una pila de registros de activación para registrar el camino de celdas desde *fuelle* hasta la celda que se está examinando en ese momento, se pueden usar los campos apuntadores del camino para contener el camino mismo. Esto es, cada celda del camino, excepto la última, contiene en el campo *derecha* o *izquierda* un apuntador a su *predecesor*, la celda más cercana a *fuelle*. Se describirá el algoritmo de Deutsch-Schorr-Waite con un campo extra de un solo bit, llamado *atrás*, que es de un tipo enumerado (*I, D*), e informa si el campo izquierdo o el derecho apunta al predecesor. Más tarde, se evaluará la forma de almacenar la información contenida en *atrás* en el campo *patrón*, sin necesidad de espacio adicional en las celdas.

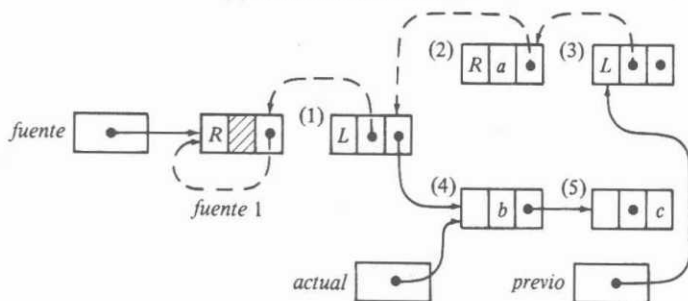
El nuevo procedimiento para búsqueda en profundidad no recursiva, que se llama *bpfnr*, se vale de un apuntador, *actual*, a la celda actual, y un apuntador, *previo*, al predecesor de la celda actual. La variable *fuelle* apunta a una celda *fuelle* que

tiene un apuntador sólo en su campo derecho †. Antes de marcar se asigna valor inicial a *fuentel* para tener *atrás* = *D*, y su campo derecho apuntando a sí mismo. A la celda que suele apuntar *fuentel*, apunta ahora *actual*, y a *fuentel* ahora *previo*. Se detiene la operación de marcado ocurre cuando *actual* = *previo*, lo cual sólo ocurre cuando ambos apuntan a *fuentel*, y se ha revisado por completo la estructura.

Ejemplo 12.3. La figura 12.6(b) muestra una posible estructura que emana de *fuelle*. Si hace una búsqueda primera de profundidad en la estructura, se visitan (1), (2), (3) y (4), en ese orden. La figura 12.6(b) muestra las modificaciones hechas a los apuntadores cuando la celda actual es (4). Se muestra el valor del campo *atrás*, aunque no sucede lo mismo con los campos *marca* y *patrón*. El camino actual va de (4)



(a) Estructura de una celda



(b) Situación en la que la celda (4) es la actual

Fig. 12.6. Uso de apuntadores para representar el camino de regreso a *fuelle*.

a (3) a (2) y a (1), de vuelta a *fuelle*; está representado por líneas (apuntadores) de puntos. Por ejemplo, la celda (1) tiene *atrás* = *I*, ya que el campo *izquierda* de (1), que en la figura 12.6 contiene un apuntador a (2), está apuntando hacia atrás, en vez de hacia adelante a lo largo del camino; sin embargo, se restaurará ese apuntador cuando la búsqueda en profundidad por fin regrese de la celda (2) a la (1). Análogamente, en la celda (2), *atrás* = *D*, y el campo derecho de (2) apunta hacia atrás en el camino hacia (1), en vez de dirigirse a (3), como lo hacía en la figura 12.6(a). □

† Esta incomodidad es necesaria debido a las peculiaridades de Pascal.

Son tres los pasos básicos para realizar la búsqueda en profundidad:

1. *Avanzar*. Si se determina que la celda actual tiene uno o más apuntadores no nulos, se avanza al primero de ellos, esto es, se sigue el apuntador en *izquierda*, pero si no lo hay, se sigue el apuntador en *derecha*. Por «avanzar» se entiende hacer que la celda apuntada se convierta en la celda actual y, la celda actual, en la anterior. Para ayudar a encontrar el camino de regreso, se hace que el apuntador recién seguido apunte a la celda anterior. Esos cambios se muestran en la figura 12.7(a), en el supuesto de que se siga al apuntador izquierdo. En esa figura, los apuntadores anteriores se representan con líneas de trazo continuo, y los nuevos, con líneas de puntos.
2. *Conmutador*. Si se determina que las celdas siguientes a la actual ya se han revisado (por ejemplo, la celda actual puede tener sólo átomos, puede estar marcada, o se pudo haber «replegado» a la celda actual desde la apuntada por el campo *derecha* de la actual), se consulta el campo *atrás* de la celda anterior. Si ese valor es *I*, y el campo *derecha* de la celda previa contiene un apuntador no nulo a alguna celda *C*, se hace que *C* se convierta en la celda actual, y la identidad de la celda anterior no se cambia. Sin embargo, el valor de *atrás* en la celda anterior se hace *D*, y el apuntador izquierdo en esa celda recibe su valor correcto; esto es, se hace que apunte a la celda actual anterior. Para mantener el camino de regreso a *fuentes* desde la celda anterior, se hace que el apuntador a *C* en el campo *derecha* apunte hacia donde apuntaba izquierda. La figura 12.7(b) muestra esos cambios.
3. *Replegar*. Si se determina, como en (2), que las celdas que parten de la actual se han recorrido, pero el campo *atrás* de la celda anterior es *D*, o es *I*, pero el campo derecho contiene un átomo o un apuntador nulo, entonces se han revisado todas las celdas que parten de la anterior. Se efectúa el repliegue haciendo que la celda anterior sea la actual y que la celda siguiente en el camino de la celda anterior a *fuentes* sea la nueva celda anterior. Esos cambios se muestran en la figura 12.7(c), en el supuesto de que *atrás* = *D* en la celda anterior.

Una coincidencia fortuita es que cada paso de la figura 12.7 se puede considerar como la rotación simultánea de tres apuntadores. Por ejemplo, en la figura 12.7(a), se reemplazan simultáneamente (*previo*, *actual*, *actual* † *izquierda*) por (*actual*, *actual* † *izquierda*, *previo*), respectivamente. Debe subrayarse la simultaneidad: la ubicación de *actual* † *izquierda* no cambia al asignar un valor nuevo a *actual*. Para realizar esas modificaciones a los apuntadores es útil contar con un procedimiento *rotar*, que se muestra en la figura 12.8. Obsérvese en especial que el paso de parámetros por referencia asegura que las ubicaciones de los apuntadores se establezcan antes de cambiar ningún valor.

Ahora se considera el diseño del procedimiento no recursivo *bpfnr* para hacer el marcado. Este procedimiento es uno de esos raros procesos que son más fáciles de entender cuando se escriben con etiquetas y proposiciones *goto*. En especial, existen dos «estados» del procedimiento, «avance» representado por la etiqueta 1, y «repliegue», representado por la etiqueta 2. Se introduce inicialmente el primer estado, y también siempre que se pasa a una celda nueva, por un paso de avance o uno de

conmutación. En este estado, se intenta otro paso de avance, y sólo se efectúa un repliegue o una conmutación si existe un bloqueo, que se puede deber a dos razones: 1) la celda recién alcanzada ya está marcada, o 2) no hay apuntadores no nulos en la celda. Cuando hay un bloqueo, se cambia al segundo estado o «replegado».

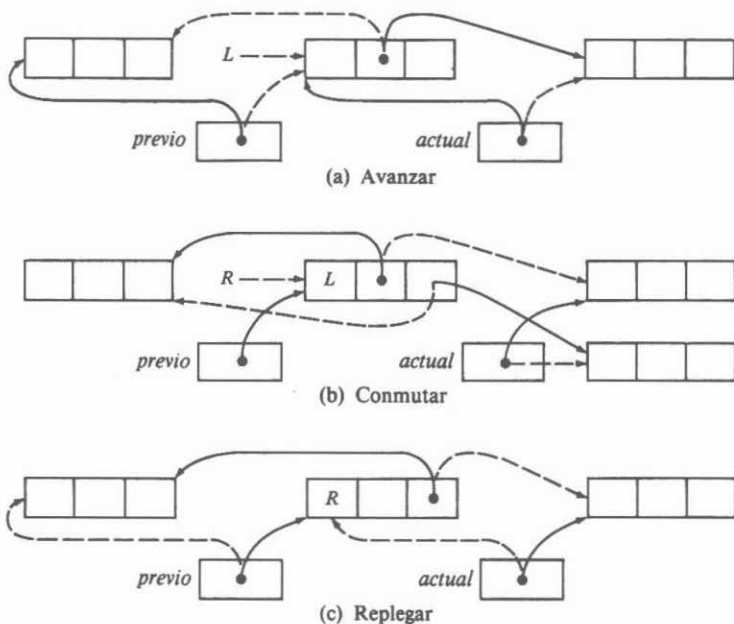


Fig. 12.7. Tres pasos básicos.

```

procedure rota ( var p1, p2, p3: ↑ tipo_celda );
  var
    temp: ↑ tipo_celda;
  begin
    temp := p1;
    p1 := p2;
    p2 := p3;
    p3 := temp
  end; { rota }

```

Fig. 12.8. Procedimiento de modificación de apuntadores.

El segundo estado se alcanzará cuando ocurra un repliegue o cuando no sea posible permanecer en estado de avance porque hay un bloqueo. En el estado de repliegue, se prueba si ya se ha replegado hasta la celda ficticia *fuentes1*. Como se analizó antes, se reconocerá esta situación porque *previo* = *actual*, en cuyo caso se pasa al estado 3. De otra forma, se decide un repliegue para permanecer en ese estado o se pasa al estado de avance. El código de *bpfnr* se muestra en la figura 12.9; este código utiliza las funciones *bloque_izq*, *bloque_der* y *bloque*, que prueban si los campos izquierdo o derecho de una celda, o ambos, tienen un átomo o un apuntador nulo, *bloque* también prueba si hay una celda marcada.

```

function bloque_izquierdo ( celda: tipo_celda): boolean;
{ prueba si el campo izquierda es un átomo o un apuntador nulo }
begin
  with celda do
    if (patrón = PP) or (patrón = PA) then
      if izquierda <> nil then return (false);
    return (true)
  end; { bloque_izquierdo }
function bloque_derecho ( celda : tipo_celda ): boolean;
{ prueba si el campo derecha es un átomo o un apuntador nulo }
begin
  with celda do
    if (patrón = PP) or (patrón = AP) then
      if derecha <> nil then
        return (false);
      return (true)
    end; { bloque_derecho }
function bloque ( celda : tipo_celda ): boolean;
{ prueba si la celda está marcada o no contiene un apuntador no nulo s }
begin
  if (celda.marca = true) or bloque_izquierdo(celda) and bloque_derecho
    (celda) then
    return (true)
  else
    return (false)
  end; { bloque }
procedure bpfnr; { marca las celdas accesibles desde fuentes }
var
  actual, previo: ↑tipo_celda;
begin { asignación de valor inicial }
  actual := fuentes1↑.derecha; { celda apuntada por fuentes1 }
  previo := fuentes1; { previo apunta a fuentes1 }
  fuentes1↑.atrás := D;
  fuentes1↑.derecha := fuentes1; { fuentes1 se apunta a sí mismo }
  fuentes1↑.marca := true;

```

```

estado 1: { intenta avanzar }
  if bloque(actual↑) then begin { prepara el repliegue }
    actual↑.marca := true;
    goto estado2
  end
  else begin { marca y avanza }
    actual↑.marca := true;
    if bloque_izquierdo(actual↑) then begin { sigue al apuntador derecho }
      actual↑.atrás := D;
      rota(previo, actual, actual↑.derecha); { realiza los cambios
        de la figura 12.7(a), pero siguiendo al apuntador derecho }
      goto estado1
    end
    else begin { sigue al apuntador izquierdo }
      actual↑.atrás := I;
      rota(previo, actual, actual↑.izquierda);
      { realiza los cambios de la figura 12.7(a) }
      goto estado1
    end
  end
end;

estado2: { termina, repliega o conmuta }
  if previo = actual then { termina }
    goto estado3
  else if (previo↑.atrás = I) and
    not bloque_derecho(previo↑) then begin { conmuta }
    previo↑.atrás := D;
    rota(previo↑.izquierda, actual, previo↑.derecha);
    { realiza los cambios de la figura 12.7(b) }
    goto estado1
  end
  else if previo↑.atrás = D then { repliega }
    rota (previo, previo↑.derecha, actual)
    { realiza los cambios de la figura 12.7(c) }
  else { previo↑.atrás = I }
    rota(previo, previo↑.izquierda, actual);
    { realiza los cambios de la figura 12.7(c), pero con el campo
      izquierda de la celda previa comprendido en el camino }
    goto estado2
  end;

estado3: { poner aquí el código para enlazar celdas no marcadas en la
  lista de espacio disponible }
end; { bpfnr }

```

Fig. 12.9. Algoritmo no recursivo para marcar celdas.

Algoritmo Deutsch-Schorr-Waite sin un bit extra para el campo *atrás*

Es posible, aunque poco probable, que el bit extra utilizado en las celdas por el campo *atrás* haga que las celdas necesiten un byte extra, o incluso de una palabra adicional. En tal caso, es bueno saber que en realidad no se necesita el bit extra, al menos si se programa en un lenguaje que, a diferencia de Pascal, permita utilizar los bits del campo *patrón* para propósitos distintos de los declarados: designadores del formato de registro variante. El «truco» consiste en observar que si se usa el campo *atrás*, como su celda está en el camino de vuelta a *fuentes*, los valores posibles del campo *patrón* están restringidos. Por ejemplo, si *atrás* = *I*, entonces se sabe que el patrón debe ser *PP* o *PA*, pues es obvio que el campo *izquierda* tiene un apuntador. Lo mismo sucede cuando *atrás* = *D*. Así, si se dispone dos bits para representar *patrón* y (cuando sea necesario) *atrás*, se puede codificar la información necesaria como en la figura 12.10, por ejemplo.

Debe observarse que en el programa de la figura 12.9, siempre se sabe si se está usando *atrás*, para saber qué interpretación de la figura 12.10 es aplicable. Tan sólo cuando *actual* apunta a un registro, el campo *atrás* en ese registro no se usa; cuando *previo* lo apunta, entonces sí. Por supuesto, cuando se mueven esos apuntadores, es necesario ajustar los códigos; por ejemplo, si *actual* apunta a una celda con los bits 10, que se interpretan de acuerdo con la figura 12.10 como *patrón* = *AP*, y se decide avanzar, de modo que *previo* apuntará ahora a esta celda, se fija *atrás* = *D*, pues sólo el campo derecho contiene un apuntador, y los bits apropiados son 11. Obsérvese que si el patrón fuera *AA*, que no tiene representación en la columna central de la figura 12.10, no se querrá que *previo* apunte a la celda, pues no hay apuntadores a seguir en un movimiento de avance.

Código	En el camino hacia <i>fuentes</i> 1	Fuera del camino
00	<i>atrás</i> = <i>I</i> , <i>patrón</i> = <i>PP</i>	<i>patrón</i> = <i>PP</i>
01	<i>atrás</i> = <i>I</i> , <i>patrón</i> = <i>PA</i>	<i>patrón</i> = <i>PA</i>
10	<i>atrás</i> = <i>D</i> , <i>patrón</i> = <i>PP</i>	<i>patrón</i> = <i>AP</i>
11	<i>atrás</i> = <i>D</i> , <i>patrón</i> = <i>AP</i>	<i>patrón</i> = <i>AA</i>

Fig. 12.10. Interpretación de dos bits como *patrón* y *atrás*.

12.4 Asignación de almacenamiento para objetos de diferentes tamaños

Considérese ahora el manejo de una estructura dinámica, como lo presenta la figura 12.1, donde hay una colección de apuntadores a bloques asignados. Los bloques contienen datos de algún tipo. En la figura 12.1, por ejemplo, los datos son cadenas de caracteres. Aunque el tipo de los datos almacenados en la estructura dinámica no tiene por qué ser cadenas de caracteres, se supone que los datos no tienen apuntadores a localidades de la estructura dinámica.

El problema de la administración de estructuras dinámicas tiene aspectos que facilitan y también dificultan su mantenimiento en comparación con las estructuras

Una consecuencia interesante de las suposiciones anteriores es que los bloques deben ser capaces de almacenar datos, algunas veces (cuando están en uso), y apun-tadores, en otras (cuando están en desuso), exactamente en el mismo lugar, con lo que es imposible o muy incómodo escribir programas que manipulen bloques de esta clase en Pascal o cualquier otro lenguaje fuertemente orientado a tipos. Así, esta sección debe ser discursiva por necesidad; sólo pueden escribirse programas en seu-do-Pascal, nunca programas reales en Pascal. Sin embargo, no hay ningún problema para escribir programas que hagan las cosas descritas en lenguaje ensamblador o en la mayoría de los lenguajes de programación de sistemas, como C.

Fragmentación y compactación de bloques vacíos

Para ver uno de los problemas especiales que se presentan en el manejo de estruc-turas dinámicas, supongáse que la variable Y de la figura 12.11 cambia, de forma que el bloque que representa a Y debe devolverse al espacio disponible. Es más fácil insertar el bloque nuevo al principio de la lista de disponibles, como se sugiere en la figura 12.12. En esa figura se observa un caso de *fragmentación*, la tendencia a representar grandes áreas vacías en la lista de espacio disponible por medio de «frag-mentos», esto es, varios bloques pequeños formando el total. En el caso en cuestión, los últimos 2300 bytes de la estructura dinámica de la figura 12.12 están vacíos, pero el espacio está dividido en tres bloques de 1000, 600 y 700 bytes, y esos bloques no están siquiera, en orden consecutivo en la lista de disponibles. Sin alguna forma de recolección de basura, sería imposible satisfacer una petición de, por ejemplo, un blo-que de 2000 bytes.

Es obvio que cuando se devuelve un bloque a la lista de disponibles es conve-niente observar los bloques inmediatamente a la izquierda y a la derecha del bloque que se está haciendo disponible. Encontrar el bloque de la derecha es sencillo; si el bloque que se está recuperando empieza en la posición p y el contador vale c , el blo-que de la derecha empieza en la posición $p + c$. Si se conoce p (por ejemplo, el apun-tador Y de la Fig. 12.11 contiene el valor p del bloque hecho disponible en la Fig. 12.12), basta con leer los bytes que empiezan en la posición p , tantos como se re-quieran para contener c , y obtener el valor c . Del byte $p + c$, se salta el campo del

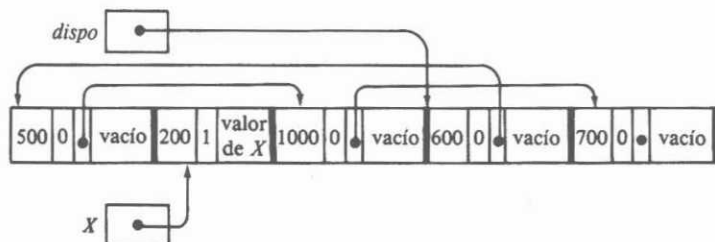


Fig. 12.12. Después de devolver el bloque de Y .

contador para encontrar el bit que indica si el bloque está vacío o no. Si está vacío, los bloques que empiezan en p y en $p + c$ pueden combinarse.

Ejemplo 12.4. Supóngase que la estructura dinámica de la figura 12.11 empieza en la posición 0. Entonces, el bloque de Y que se está devolviendo empieza en el byte 1700, así que $p = 1700$ y $c = 600$. El bloque que empieza en $p + c = 2300$ también está vacío, por lo que se pueden combinar en un solo bloque que empieza en 1700 con un contador de 1300, la suma de los contadores de los dos bloques. □

Sin embargo, no es tan fácil fijar la lista de disponibles después de combinar los bloques. Se puede crear el bloque combinado agregando tan sólo el contador del segundo bloque a c . Sin embargo, el segundo bloque permanecerá enlazado en la lista de disponibles y deberá separarse, para lo cual es necesario encontrar el apuntador a ese bloque desde su predecesor en la lista de disponibles. Se presentan varias estrategias, pero ninguna se recomienda en especial.

1. Recorrer la lista hasta encontrar un apuntador con valor $p + c$. Este apuntador debe estar en el bloque anterior al que se ha combinado con su vecino. Reemplazar el apuntador encontrado por el apuntador del bloque de $p + c$. Esto, en efecto, elimina el bloque que empieza en la posición $p + c$ de la lista de disponibles. Su espacio queda disponible, por supuesto; es parte del bloque que empieza en p . En promedio, habrá que revisar la mitad de la lista de disponibles, así que el tiempo requerido es proporcional a esa longitud.
2. Usar una lista doblemente enlazada para el espacio disponible. Entonces, el bloque predecesor puede encontrarse con rapidez y el bloque en $p + c$ se puede eliminar de la lista. Este enfoque requiere un tiempo constante, independiente de la longitud de la lista de disponibles, pero requiere espacio adicional para otro apuntador en cada bloque vacío, incrementando así el tamaño mínimo de un bloque para contener un contador, un bit vacío/lleño, y dos apuntadores.
3. Conservar clasificada toda la lista de espacio disponible de acuerdo con la posición. Entonces, se sabe que el bloque de la posición p es el predecesor en la lista del bloque en $p + c$, y la manipulación del apuntador necesaria para eliminar el segundo bloque puede efectuarse en un tiempo constante. Sin embargo, la inserción de un bloque disponible nuevo requiere revisar en promedio la mitad de la lista, por lo que no es más eficiente que el método (1).

De los tres métodos, el primero y el tercero requieren un tiempo proporcional a la longitud de la lista de disponibles para devolver un bloque al espacio disponible y combinarlo con su vecino derecho, si está vacío. Este tiempo puede ser prohibitivo o no, dependiendo de lo larga que sea la lista y del tiempo total del programa se consume en la manipulación de la estructura dinámica. El segundo método —doble enlace para la lista de disponibles— tiene sólo el defecto de incrementar el tamaño mínimo de los bloques. Lamentablemente, cuando se considera cómo combinar un bloque devuelto con su vecino de la izquierda, como en los otros métodos, se observa que el doble enlace no ayuda a encontrar vecinos izquierdos en menos tiempo que el requerido para recorrer la lista de disponibles.

Encontrar un bloque inmediatamente a la izquierda del actual no es fácil. La po-

sición p de un bloque y de su contador c determinan la posición del bloque de la derecha, pero esto no sirve como guía para encontrar el principio del bloque izquierdo. Es necesario encontrar un bloque vacío que empiece en alguna posición p_1 y tenga un contador c_1 tal que $p_1 + c_1 = p$. Para esto hay tres posibles estrategias.

1. Revisar la lista de disponibles buscando un bloque en la posición p_1 y con contador c_1 donde $p_1 + c_1 = p$. Esta operación lleva un tiempo proporcional a la longitud de la lista de disponibles.
2. Conservar un apuntador en cada bloque (usado o no) indicando la posición del bloque de la izquierda. Este enfoque permite encontrar el bloque izquierdo en un tiempo constante; se prueba si está vacío, en cuyo caso se combina con el bloque en cuestión. Se puede encontrar el bloque en la posición $p + c$ y hacer que apunte al principio del bloque nuevo, para poder mantener esos apuntadores a la izquierda $\dagger$.
3. Mantener clasificada la lista de disponibles de acuerdo con la posición. Entonces el bloque vacío de la izquierda se encuentra al insertar en la lista el bloque vaciado recientemente, y sólo es necesario revisar, usando la posición y el contador del bloque vacío anterior, que no se interponga ningún bloque no vacío.

Al igual que con la intercalación de bloques vacíos recientes con el bloque de la derecha, el primero y tercer enfoques para la búsqueda e intercalación con el bloque de la izquierda requieren un tiempo proporcional a la longitud de la lista de disponibles. El método (2) también requiere tiempo constante, pero tiene una desventaja sobre los problemas relativos a la lista de disponibles doblemente enlazada (ya sugerida en relación con la búsqueda de los bloques vecinos de la derecha). Mientras que el doble enlace de los bloques vacíos aumenta el tamaño mínimo de los bloques, no puede decirse que este enfoque desperdicie espacio, ya que sólo los bloques que no se usan para almacenar datos son los que se enlazan. Sin embargo, apuntar a los vecinos izquierdos requiere un apuntador en los bloques utilizados y en los no utilizados, y con justicia puede acusársele de consumir espacio. Si el tamaño promedio de los bloques es de cientos de bytes, el espacio extra para un apuntador puede ser despreciable. Por otro lado, el espacio adicional puede ser prohibitivo si el bloque típico tiene sólo 10 bytes de longitud.

Para resumir las implicaciones de estas exploraciones en cuanto a cómo combinar bloques vacíos recientes con vecinos vacíos, hay tres enfoques para tratar la fragmentación.

1. Usar uno de varios enfoques, como puede ser conservar clasificada la lista de disponibles, que requiere un tiempo proporcional a la longitud de la lista cada vez que un bloque queda sin utilizar, pero permite encontrar y combinar vecinos vacíos.
2. Usar una lista de espacio disponible doblemente enlazada cada vez que un bloque quede sin utilizar, y también utilizar apuntadores a los vecinos izquierdos de todos los bloques, estén o no disponibles, para combinar vecinos vacíos en un tiempo constante.

$\dagger$ Como ejercicio, se debe descubrir cómo mantener los apuntadores cuando un bloque se divide en dos; se toma una parte para un elemento de datos nuevo, mientras que la otra permanece vacía.

3. No hacer nada explícito para combinar vecinos vacíos. Cuando no sea posible encontrar un bloque tan grande como para contener datos nuevos, revisar los bloques de izquierda a derecha, combinando vecinos vacíos y después crear una lista nueva de disponibles. Un bosquejo del programa que hace esto se muestra en la figura 12.13.

```

(1)  procedure combina;
      var
(2)    p, q: apuntadores a bloques;
        { p indica el extremo izquierdo del bloque vacío que se está
          acumulando; q indica un bloque a la derecha de p que se
          incorporará en el bloque p si está vacío }
      begin
(3)    p := bloque más a la izquierda de la estructura dinámica;
(4)    vaciar la lista de disponibles;
(5)    while p < extremo derecho de la estructura dinámica do
(6)      if p apunta a un bloque lleno con contador c then
(7)        p := p + c; { saltar los bloques llenos }
(8)      else begin { p apunta al principio de una secuencia de
        bloques vacíos; deben combinarse }
(9)        q := p + c; { asignar el valor inicial q al siguiente bloque }
(10)       while q apunta a un bloque vacío con un contador, d, y q <
        extremo derecho de la estructura dinámica do begin
(11)         agregar d al contador del bloque apuntado por p;
(12)         q := q + d
        end
(13)       inserta el bloque apuntado por p en la lista de disponibles;
(14)       p := q;
      end
end; { combina }

```

Fig. 12.13. Combinación de bloques vacíos adyacentes.

Ejemplo 12.5. Como ejemplo, considérese el programa de la figura 12.13, aplicado a la estructura dinámica de la figura 12.12. Supóngase que el byte de más a la izquierda de la estructura dinámica es cero, así que inicialmente, $p = 0$. Como $c = 500$ para el primer bloque, a q se le asigna un valor inicial $p + c = 500$. Cuando el bloque que empieza en 500 está lleno, el ciclo de las líneas (10) a (12) no se ejecuta y el bloque que consta de los bytes 0 a 499 se coloca en la lista de disponibles, haciendo que *dispo* apunte al byte 0 y poniendo un apuntador *nil* en el lugar designado en ese bloque (después del contador y el bit vacío/lleno). Después, se asigna a p el valor 500 en la línea (14), y se incrementa a 700 en la línea (7). El apuntador q toma el valor 1700 en la línea (9), y después, 2300 y 3000 en la línea (12), al tiempo que 600 y 700 se agregan al contador 1000 en el bloque que empieza en 700. Cuando q excede del byte de más a la derecha, 2999, el bloque que empieza en 700, que ahora tiene

el contador 2300, se inserta en la lista de disponibles. Entonces, en la línea (14), p se ajusta en 3000 y el ciclo externo finaliza en la línea (5). □

El número total de bloques y el número de bloques disponibles puede no ser muy diferente, y probablemente la frecuencia con que se puede encontrar que no hay bloques vacíos bastante grandes sea baja, se cree que el método (3), combinar bloques vacíos adyacentes sólo cuando se termine el espacio adecuado, es superior a (1) en una situación real. El método (2) es un posible competidor, pero considera los requisitos de espacio adicional y el hecho de que se requiere un tiempo extra cada vez que se inserta o elimina un bloque de la lista de disponibles, se cree que (2) será preferible a (3) en circunstancias muy raras, y tal vez se pueda olvidar.

Selección de bloques disponibles

Se ha tratado con detalle qué debe suceder cuando deja de necesitarse un bloque y se puede devolver al espacio disponible. Existe también el proceso inverso de proporcionar bloques para contener nuevos datos. Es evidente que se debe seleccionar algún bloque disponible y usarlo parcial o totalmente para contener los datos nuevos. Hay dos temas a tener en cuenta. Primero, ¿qué bloque vacío se selecciona? Segundo, si es necesario usar sólo parte de un bloque seleccionado, ¿qué parte se usa?

La segunda pregunta es fácil de aclarar. Si se utiliza un bloque con contador c y se requieren $d < c$ bytes de ese bloque, se escogen los últimos d bytes. De esta forma, sólo es necesario reemplazar c por $c-d$, y el bloque vacío restante puede permanecer en la lista de disponibles †. □

Ejemplo 12.6. Supóngase que se requieren 400 bytes para la variable W en la situación representada en la figura 12.12. Podría decidirse tomar los 400 bytes finales de los 600 que existen en el primer bloque de la lista de disponibles. Tal situación se muestra en la figura 12.14.

La selección de un bloque para colocar datos nuevos no es fácil, debido a que existen conflictos entre las metas de tales estrategias. Se desea, por un lado, poder tomar con rapidez un bloque vacío en el cual quepan los datos y, por otro, hacer una selección de un bloque vacío que reduzca la fragmentación. Hay dos estrategias que representan extremos en el espectro conocidas como «primer ajuste» y «mejor ajuste», y se describen a continuación.

1. *Primer ajuste.* Para seguir la estrategia de *primer ajuste*, cuando se necesita un bloque de tamaño d , se revisa la lista de disponibles desde el principio hasta llegar a un bloque de tamaño $c \geq d$. Utilícense las últimas d palabras en ese bloque, como se describió antes.
2. *Mejor ajuste.* Para seguir la estrategia de *mejor ajuste*, cuando se necesita un bloque de tamaño d , se examina toda la lista de disponibles para encontrar el bloque de tamaño mínimo d que sea lo más cercano posible a d . Se toman las últimas d palabras de ese bloque.

† Si $c - d$ es tan pequeño que un contador y un apuntador no caben, hay que utilizar el bloque completo y eliminarlo de la lista de disponibles.

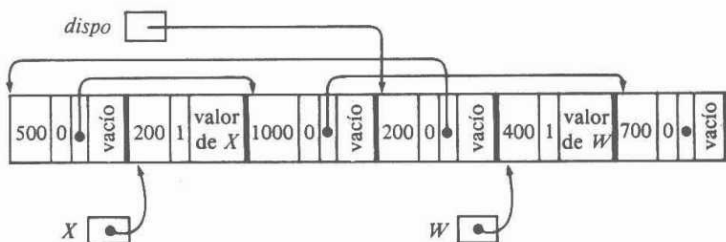


Fig. 12.14. Configuración de la memoria.

Pueden hacerse algunas observaciones acerca de estas estrategias. La estrategia de mejor ajuste es bastante más lenta que la de primer ajuste, ya que con esta última se puede esperar encontrar, en promedio, un bloque suficiente con rapidez, mientras que con la de mejor ajuste es necesario recorrer toda la lista de disponibles. La estrategia de mejor ajuste puede acelerarse si se mantienen listas de bloques disponibles de acuerdo con varias gamas de tamaños. Por ejemplo, se puede mantener una lista de bloques disponibles con longitud entre 1 y 16 bytes †, entre 17 y 32, entre 33 y 64, y así sucesivamente. Esta «mejora» no beneficia en forma apreciable la estrategia de primer ajuste, y, de hecho, puede alentarla si las estadísticas de los tamaños de bloques son malas. (Compárese la búsqueda del primer bloque de tamaño mayor que 32 en la lista de disponibles completa y en la lista de bloques de tamaño 17 a 32, por ejemplo.) Una última observación es que un espectro de estrategias se puede definir entre las dos planteadas aquí, buscando el mejor ajuste entre los primeros k bloques disponibles para algún tamaño fijo k .

La estrategia de mejor ajuste parece reducir la fragmentación en comparación con la de primer ajuste, en el sentido de que la primera tiende a producir «fragmentos» muy pequeños, es decir, bloques abandonados. Y aunque el número de esos fragmentos es casi el mismo que para el primer ajuste, tienden a ocupar un área menor. Sin embargo, el mejor ajuste no tiende a producir «fragmentos» de tamaño medio. En cambio, los bloques disponibles tienden a ser fragmentos muy pequeños o bloques devueltos al espacio disponible. Como consecuencia, hay secuencias de peticiones que la estrategia de primer ajuste puede satisfacer y la de mejor ajuste no, y viceversa.

Ejemplo 12.7. Supóngase, como en la figura 12.12, que la lista de disponibles consta de los bloques de tamaño 600, 500, 1000 y 700, en ese orden. Si se emplea la estrategia de primer ajuste y se hace una petición de un bloque de tamaño 400, se obtiene el bloque de tamaño 600 que es el primero de la lista en el cual cabe un bloque de tamaño 400. La lista de disponibles tiene ahora bloques de tamaño 200, 500, 1000 y 700. Así que es imposible satisfacer de inmediato tres peticiones de bloques de tamaño 600 (aunque se puede hacer después de combinar bloques vacíos adyacentes o al recorrer bloques utilizados sobre la estructura dinámica).

† En realidad, hay un tamaño de bloque mínimo mayor que 1, puesto que los bloques deben contener un apuntador, un contador y un bit vacío/lleño si se van a encadenar a una lista de disponibles.

No obstante, si se aplicara la estrategia de mejor ajuste con la lista de disponibles 600, 500, 1000 y 700, y llegara la petición de 400, podría colocarse donde ajusta mejor, esto es, en el bloque de 500, dejando una lista de bloques disponibles de 600, 100, 1000 y 700. En este caso, se pueden satisfacer tres peticiones de bloques de tamaño 600 sin necesidad de recurrir a la reorganización del almacenamiento.

Por otro lado, hay situaciones donde, al empezar con la lista 600, 500, 1000, 700 de nuevo, la estrategia de mejor ajuste puede fallar, mientras que la de primer ajuste puede funcionar sin necesidad de reorganización del almacenamiento. Dada la petición de 400 bytes, el mejor ajuste puede, igual que antes, dejar la lista en 600, 100, 1000 y 700, mientras que el primer ajuste la deja en 200, 500, 1000 y 700. Supóngase que las dos peticiones siguientes son de 1000 y 700, de manera que cualquier estrategia asignaría completamente los dos últimos bloques vacíos, dejando 600 y 100 en el caso del mejor ajuste, y 200 y 500, en el caso del primer ajuste. Ahora bien, el primer ajuste puede satisfacer peticiones de bloques de tamaño 200 y 500, mientras que el mejor ajuste obviamente no podrá. □

12.5 Sistemas de manejo de memoria por afinidades (buddy systems)

Hay una familia de estrategias para mantener una estructura dinámica que evite parcialmente los problemas de fragmentación y distribución difícil de bloques vacíos. Esas estrategias, llamadas «sistemas de manejo de memoria por afinidades», consumen poco tiempo al combinar bloques vacíos adyacentes. El inconveniente es que los bloques llegan en un surtido limitado de tamaños, así que se puede desperdiciar algún espacio colocando datos en un bloque más grande de lo necesario.

La idea central de todos los sistemas por afinidades es que los bloques son sólo de ciertos tamaños; por ejemplo, $s_1 < s_2 < s_3 < \dots < s_k$ son todos los tamaños entre los cuales pueden encontrarse los bloques. Algunas selecciones comunes para la secuencia $s_1, s_2, \dots$ son 1, 2, 4, 8, ... (el *sistema de afinidades exponencial*) y 1, 2, 3, 5, 8, 13, ... (el *sistema de afinidades de Fibonacci*, donde $s_{i+1} = s_i + s_{i-1}$). Todos los bloques vacíos de tamaño s_i están enlazados en una lista, y hay un arreglo de encabezados de lista de disponibles, una para cada tamaño s_i permitido †. Si se requiere un bloque de tamaño d para un grupo de datos nuevo, se escoge un bloque disponible de tamaño s_i tal que $s_i \geq d$, pero $s_{i-1} < d$, esto es, el tamaño más pequeño permitido en el cual caben los datos nuevos.

Surgen algunas dificultades cuando no existen bloques vacíos del tamaño deseado s_i . En ese caso, es necesario encontrar un bloque de tamaño s_{i+1} y dividirlo en dos, uno de tamaño s_i y el otro de tamaño $s_{i+1} - s_i$ ††. El sistema de afinidades impone la restricción de que $s_{i+1} - s_i$ sea alguna s_j , para $j \leq i$. Ahora se ve la forma en

† Puesto que los bloques vacíos deben contener apuntadores (y, como se verá, otra información también), en realidad no se inicia la secuencia de tamaños permitidos en 1, sino en algún número apropiado más grande en la secuencia, como 8 bytes.

†† Por supuesto, si no hay bloques vacíos de tamaño s_{i+1} , se crea uno dividiendo un bloque de tamaño s_{i+2} , y así sucesivamente. Si no existen bloques de ningún tamaño mayor, en realidad no hay espacio y es necesario reorganizar la estructura dinámica como en la sección siguiente.

que se puede restringir la selección de valores para las s_i . Si se hace que $j = i - k$, para alguna $k \geq 0$, dado que $s_{i+1} - s_i = s_{i-k}$, se sigue que

$$s_{i+1} = s_i + s_{i-k} \quad (12.1)$$

La ecuación (12.1) es aplicable cuando $i > k$, y junto con los valores para $s_1, s_2, \dots, s_k$, determina completamente a $s_{k+1}, s_{k+2}, \dots$. Por ejemplo, si $k = 0$, (12.1) queda como

$$s_{i+1} = 2s_i \quad (12.2)$$

empezando con $s_1 = 1$ en (12.2), se obtiene la secuencia exponencial 1, 2, 4, 8, Por supuesto, no importa el valor inicial de s_1 ; las s crecerán exponencialmente en (12.2). Otro ejemplo, si $k = 1$, $s_1 = 1$ y $s_2 = 2$, (12.1) se transforma en

$$s_{i+1} = s_i + s_{i-1} \quad (12.3)$$

esta ecuación define la sucesión de Fibonacci: 1, 2, 3, 5, 8, 13,

Para cualquier valor de k escogido en (12.1), habrá un *sistema de afinidades de orden k -ésimo*. Para cualquier k , la secuencia de tamaños permitidos crece exponencialmente; esto es, la razón s_{i+1}/s_i se aproxima a alguna constante mayor que uno. Por ejemplo, para $k = 0$, s_{i+1}/s_i es exactamente 2. Para $k = 1$ la razón se aproxima a la «razón dorada» $(\sqrt{5}+1)/2 = 1.618$, y la razón decrece conforme k crece, pero nunca es menor que 1.

Distribución de bloques

En el sistema de afinidades de orden k -ésimo, cada bloque de tamaño s_{i+1} se puede considerar formado por un bloque de tamaño s_i y otro de tamaño s_{i-k} . Para ser más específicos, supóngase que el bloque de tamaño s_i está a la izquierda (en las posiciones numeradas menores) del bloque de tamaño s_{i-k} †. Si se considera la estructura dinámica como un solo bloque de tamaño s_n para alguna n grande, entonces las posiciones en las que los bloques de tamaño s_i pueden empezar están completamente determinadas.

Las posiciones en el sistema exponencial, o de orden 0-ésimo, son fáciles de determinar. Si se supone que las posiciones en la estructura están numeradas a partir de 0, un bloque de tamaño s_i empieza en cualquier posición que comience con un múltiplo de 2^i , esto es, 0, $2^i, \dots$. Más aún, cada bloque de tamaño 2^{i+1} , que empiece, por ejemplo, en $j2^{i+1}$ está compuesto de dos «grupos afines» de tamaño 2^i , que empiezan en las posiciones $(2j)2^i$, que son $j2^{i+1}$, y $(2j+1)2^i$. Así, es fácil encontrar el grupo afín de un bloque de tamaño 2^i . Si empieza en algún múltiplo par de 2^i , como $(2j)2^i$, su afín está a la derecha, en la posición $(2j+1)2^i$. Si empieza en un múltiplo impar de 2^i , por ejemplo, en $(2j+1)2^i$, su afín está a la izquierda, en $(2j)2^i$.

Ejemplo 12.8. No es simple el comportamiento de los sistemas de afinidades de orden mayor que cero. La figura 12.15 muestra el sistema de afinidades de Fibonacci

† A veces es conveniente considerar que los bloques de tamaño s_i y s_{i-k} forman un bloque de tamaño s_{i+1} como «afines»; de ahí el término «sistema de afinidades».

utilizado en una estructura dinámica de tamaño 55, con bloques de tamaños $s_1, s_2, \dots$, $s_8 = 2, 3, 5, 8, 13, 21, 34$ y 55. Por ejemplo, el bloque de tamaño 3 que empieza en 26 es afín al bloque de tamaño 5 que empieza en 21; juntos, forman el bloque de tamaño 8 que empieza en 21 y que es afín al bloque de tamaño 5 que empieza en 29. Juntos, estos conforman el bloque de tamaño 13 que empieza en 21, y así sucesivamente. $\square$

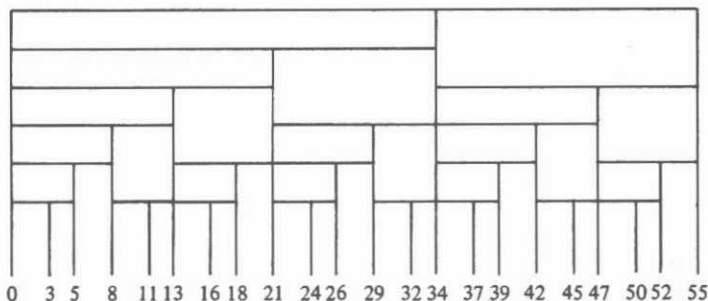


Fig. 12.15. División de una estructura dinámica según el sistema de afinidades de Fibonacci.

Asignación de bloques

Si se requiere un bloque de tamaño n , se escoge cualquiera de la lista de disponibles de tamaño s_i , donde $s_i \geq n$ o $i = 1$ o $s_{i-1} < n$, esto es, el bloque de mejor ajuste. En un sistema de afinidades de orden k -ésimo, si no hay bloques de tamaño s_i , es posible elegir un bloque de tamaño s_{i+1} o s_{i+k+1} para dividirlo, obteniendo en cualquier caso algún bloque de tamaño s_i . Si no existen bloques de esos tamaños, se crea uno aplicando la estrategia de división recursivamente para el tamaño s_{i+k} .

Sin embargo, hay una pequeña trampa. En un sistema de orden k -ésimo, puede ser que no se dividan los bloques de tamaño $s_1, s_2, \dots, s_k$, ya que de eso puede resultar un bloque de tamaño menor que s_1 . Se debe usar dicho bloque completo si no se encuentra disponible otro menor. Este problema no surge si $k = 0$, esto es, en el sistema exponencial. Puede reducirse en el sistema de Fibonacci si se empieza con $s = 1$, pero esa selección quizá no sea aceptable, puesto que los bloques de tamaño uno (byte o palabra, tal vez), pueden ser muy pequeños para contener un apuntador y un bit vacío/lleño.

Devolución de bloques al espacio disponible

Cuando un bloque queda listo para reutilizarse, se puede ver una de las ventajas de los sistemas de afinidades. Algunas veces se puede reducir la fragmentación combinando el bloque que quedó disponible recientemente con su afín, si éste está dispo-

nible también †. De hecho, si fuera el caso, se puede combinar el bloque resultante con su afín, si está vacío, y así sucesivamente. La combinación de grupos afines vacíos requiere sólo una cantidad constante de tiempo, lo que la convierte en una alternativa atractiva cuando se presentan combinaciones periódicas de bloques vacíos adyacentes, como se sugirió en la sección anterior, que requiere un tiempo proporcional al número de bloques vacíos.

El sistema exponencial hace que la localización de grupos afines sea especialmente fácil. Si se acaba de devolver el bloque de tamaño 2^i que empieza en $p2^i$, su afín está en $(p+1)2^i$ si p es par, y en $(p-1)2^i$ si p es impar.

Para un sistema de orden $k \geq 1$, la localización de afines no es tan simple. Para hacerla más fácil es necesario almacenar cierta información en cada bloque.

1. Un bit vacío/lleño, como tienen todos los bloques.
2. El índice de tamaño, un entero i tal que el bloque sea de tamaño s_i .
3. El contador de afinidad izquierdo, descrito a continuación.

En cada par de afines, uno (el afín izquierdo) está a la izquierda del otro (el afín derecho). Intuitivamente, el contador de afinidad izquierdo de un bloque dice cuántas veces consecutivas éste es todo o parte de un afín izquierdo. En un aspecto formal, toda la estructura dinámica, tratada como un bloque de tamaño s_n , tiene un contador de afinidad izquierdo igual a 0. Cuando se divide cualquier bloque de tamaño s_{i+1} , con el contador de afinidad izquierdo b , en bloques de tamaño s_i y s_{i-k} , que son los afines izquierdo y derecho, respectivamente, el afín izquierdo tiene un contador $b+1$, mientras que el derecho tiene un contador igual a 0, independiente de b . Por ejemplo, en la figura 12.15, el bloque de tamaño 3 que empieza en 0 tiene un contador de afinidad izquierdo igual a 6, y el bloque de tamaño 3 que empieza en 13 tiene un contador izquierdo igual a 2.

Además de la información anterior, los bloques vacíos, pero no los utilizados, tienen apuntadores de avance y de retroceso para la lista de disponibles del tamaño adecuado. Los apuntadores bidireccionales facilitan las combinaciones de afines, que deben eliminarse de la lista de disponibles.

La forma en que se maneja esta información es como sigue. Supóngase que k es el orden del sistema de afinidades. Cualquier bloque que empiece en la posición p con un contador de afinidad izquierdo igual a 0 es un afín derecho. Así, si tiene índice de tamaño j , su afín izquierdo es de tamaño s_{j+k} y empieza en la posición $p-s_{j+k}$. Si el contador de afinidad izquierdo es mayor que 0, entonces el bloque es un afín izquierdo de un bloque de tamaño s_{j-k} , el cual está localizado en la posición $p+s_j$.

Si se combina un afín izquierdo de tamaño s_j , teniendo un contador de afinidad izquierdo igual a b , con un afín derecho de tamaño s_{i-k} , el bloque resultante tendrá un índice de tamaño $i+1$, que empieza en la misma posición que el bloque de tamaño s_j , y tiene un contador de afinidad izquierdo $b-1$. Así, la información necesaria puede mantenerse con facilidad al combinar dos afines vacíos. Como ejercicio, se puede comprobar que la información se mantiene al dividir un bloque vacío de tamaño s_{i+1} en un bloque ocupado de tamaño s_i y uno vacío de tamaño s_{i-k} .

† Igual que en la sección anterior, se supone que un bit de cada bloque está reservado para indicar si el bloque está en uso o vacío.

Si se mantiene toda esta información, y se ligan las listas de disponibles en ambas direcciones, sólo se emplea una cantidad constante de tiempo en cada división de un bloque o se combinan los afines en un bloque más grande. Como el número de combinaciones nunca puede exceder del número de divisiones, la cantidad total de trabajo es proporcional a este número. No es difícil reconocer que la mayor parte de las solicitudes de un bloque no requieren divisiones, ya que hay disponible un bloque del tamaño correcto. Sin embargo, hay situaciones extremas en las que cada asignación requiere bastantes divisiones. El ejemplo extremo es donde se solicita repetidamente un bloque del tamaño más pequeño y después se devuelve. Si hay n tamaños diferentes, se requieren por lo menos n/k divisiones en un sistema de orden k -ésimo, las cuales estarán seguidas de n/k combinaciones cuando se devuelve el bloque.

12.6 Compactación del almacenamiento

Hay ocasiones en las que, aun después de combinar todos los bloques adyacentes, no es posible satisfacer una solicitud de un bloque nuevo. Esto puede deberse simplemente a que en el montón no hay espacio para formar un bloque del tamaño deseado. Pero es más típica una situación como la que se muestra en la figura 12.11, donde si bien hay 2200 bytes disponibles, no se puede satisfacer una solicitud de un bloque mayor de 1000. El problema es que el espacio disponible está dividido entre varios bloques no contiguos. Hay dos enfoques generales a este problema.

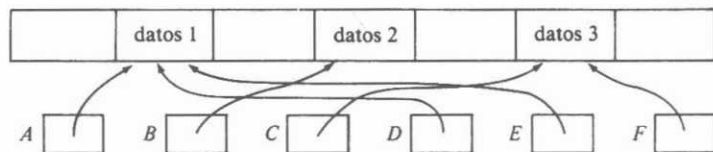
1. Lograr que el espacio disponible para el conjunto de datos pueda estar compuesto de varios bloques vacíos. Si ocurre así, también se puede requerir que todos los bloques sean del mismo tamaño y consistan en espacio para un apuntador y para los datos. En un bloque utilizado, el apuntador indica el siguiente bloque usado para los datos, y es nulo en el último bloque. Por ejemplo, si se estuvieran almacenando datos cuyo tamaño fuera casi siempre pequeño, se escogerían bloques de tamaño igual a 16 bytes, con 4 utilizados para un apuntador y 12 para los datos. Si los conjuntos de datos fueran en general más grandes, podrían escogerse bloques con varios cientos de bytes, asignando de nuevo cuatro para un apuntador y el resto para los datos.
2. Cuando falla la combinación de bloques vacíos adyacentes, y no se es capaz de proveer un bloque bastante grande, se mueven los datos por la estructura dinámica de manera que todos los bloques llenos queden en el extremo izquierdo (posición más baja), y se forme un bloque grande disponible a la derecha.

El método (1), que usa cadenas de bloques para un conjunto de datos, tiende a consumir mucho espacio. Si se escoge un tamaño de bloque pequeño, se emplea una fracción grande de espacio para «las cabeceras», los apuntadores necesarios para mantener las cadenas. Al utilizar bloques grandes, se ocupa menos espacio en las cabeceras, pero muchos bloques estarán casi desperdiciados, almacenando pocos datos. La única situación en que esta clase de enfoque es preferible, es cuando los conjuntos de datos típicos sean muy grandes. Por ejemplo, muchos sistemas de archi-

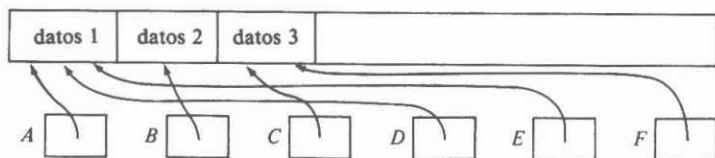
vos trabajan de esta forma, dividiendo la estructura dinámica, que por lo común es una unidad de disco, en bloques del mismo tamaño, por ejemplo 512 a 4096 bytes, dependiendo del sistema. Como muchos archivos son mucho más grandes que esos números, no se desperdicia demasiado el espacio, y los apuntadores a los bloques que conforman un archivo requieren relativamente poco espacio. La asignación de espacio bajo esta disciplina es casi directa, debido a lo que se ha aprendido en las secciones anteriores, por lo que no se analizará aquí la manera de realizarla.

Problema de la compactación

Un problema que debe enfrentarse con frecuencia es el de tomar una colección de bloques en uso, como la sugerida en la figura 12.16(a), donde cada bloque puede ser de tamaño diferente y estar apuntado por más de un apuntador, y correrlos hacia la izquierda hasta que todo el espacio disponible quede en el extremo derecho de la estructura dinámica, como se muestra en la figura 12.16(b). Como es natural, los apuntadores deben continuar apuntando a los mismos datos.



(a) Antes de la compactación



(b) Después de la compactación

Fig. 12.16. Proceso de compactación del almacenamiento.

Existen soluciones simples a este problema asignando un poco de espacio adicional en cada bloque, y se analizará otro método más complicado, pero eficiente, que no requiere espacio extra en los bloques utilizados, sólo el que se ha usado hasta ahora en cualquiera de los esquemas de manejo del espacio estudiados, es decir, un bit vacío/lleño y un contador que indique el tamaño del bloque.

Un esquema simple para la compactación consiste en revisar primero todos los bloques desde la izquierda, sean llenos o vacíos, y calcular una «dirección de avance» para cada bloque lleno. La *dirección de avance* de un bloque es su posición actual menos la suma de todo el espacio vacío que se encuentra a su izquierda, esto es, la posición a la cual habrá de pasar finalmente. Es fácil calcular la dirección de avance. Al revisar los bloques desde la izquierda, se acumula la cantidad de espacio vacío que se encuentra para sustraer esta cantidad a la posición de cada bloque visto. El algoritmo se plantea en la figura 12.17.

```

(1)  var
      p: integer; { posición del bloque actual }
      hueco: integer; { cantidad total de espacio vacío hallado
                       hasta ahora }
begin
(2)  p := extremo izquierdo de la estructura dinámica;
(4)  hueco := 0;
(5)  while p ≤ extremo derecho de la estructura dinámica do begin
      { hace que p apunte al bloque B }
(6)  if B está vacío then
(7)    hueco := hueco + contador en el bloque B
      else { B está lleno }
(8)    dirección de avance de B := p - hueco;
(9)    p := p + contador en el bloque B
      end
end;
```

Fig. 12.17. Cálculo de las direcciones de avance.

Una vez calculada la dirección de avance, se revisan todos los apuntadores al montón †. Se sigue cada apuntador hacia algún bloque *B* para reemplazar el apuntador por la dirección de avance encontrada en ese bloque. Por último, se mueven todos los bloques llenos hacia su dirección de avance. Este proceso es similar al de la figura 12.17, con la línea (8) reemplazada por

```

for i:= p to p-1 + contador en B do
  estructura dinámica [i - hueco] := estructura dinámica[i];
```

para pasar el bloque *B* a la izquierda en una cantidad igual a *hueco*. Obsérvese que el movimiento de bloques llenos, que requiere un tiempo proporcional a la cantidad de bloques en uso dentro de la estructura dinámica, probablemente dominará los demás costos de compactación.

† En el resto del texto se supone que la colección de dichos apuntadores está disponible. Por ejemplo, una realización normal de SNOBOL almacena pares que constan de un nombre de variable y un apuntador al valor de ese nombre en una tabla de dispersión, con la función de dispersión calculada a partir del nombre. Examinar la tabla completa permite visitar todos los apuntadores.

Algoritmo de Morris

F. L. Morris descubrió un método para la compactación de la estructura dinámica sin usar espacio en los bloques para la dirección de avance. No obstante, requiere un bit de marca asociado con cada apuntador y con cada bloque para indicar el final de una cadena de apuntadores. La idea esencial es crear una cadena de apuntadores que salen de una posición fija en cada bloque lleno y enlazados todos a ese bloque. Por ejemplo, en la figura 12.16(a) se observan tres apuntadores, *A*, *D* y *E*, apuntando al bloque lleno del extremo izquierdo. En la figura 12.18, se encuentra la cadena deseada de apuntadores. Una porción de los datos de tamaño igual a la de un apuntador se ha extraído del bloque y colocado al final de la cadena, donde se encontraba el apuntador *A*.

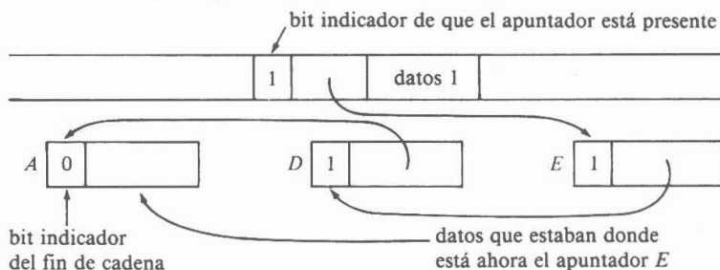


Fig. 12.18. Encadenamiento de apuntadores.

El método para crear dichas cadenas de apuntadores es como sigue. Se examinan todos los apuntadores en cualquier orden conveniente. Supóngase un apuntador *p* al bloque *B*. Si el bit de marca en el bloque *B* es cero, entonces *p* es el primer apuntador encontrado que apunta a *B*. Se colocan en *p* los contenidos de las posiciones de *B* utilizadas para la cadena de apuntadores, y se hace que esas posiciones de *B* apunten a *p*. Después, se hace el bit de marca en *B* igual a 1, indicando que ahora tiene un apuntador, y el bit de marca en *p* igual a 0, indicando el fin de la cadena de apuntadores y la presencia de los datos desplazados.

Ahora bien, al considerar el apuntador *p* al bloque *B*, el bit de marca en *B* está en uno; entonces, *B* ya tiene la cabeza de la cadena de apuntadores. Se copia el apuntador de *B* en *p*, para que *B* apunte a *p*, y se hace el bit de marca en *p* igual a 1; de este modo, *p* queda, efectivamente, en la cabeza de la cadena.

Una vez que se tienen todos los apuntadores a cada bloque enlazados en una cadena que parte desde ese bloque, se pasan los bloques llenos tan a la izquierda como sea posible, igual que en el sencillo algoritmo analizado con anterioridad. Por último, se examina cada bloque en su nueva posición y se recorre su cadena de apuntadores. Cada apuntador localizado se actualiza para que apunte al bloque en su nueva posición. Al encontrar el final de la cadena se reinstalan los datos de *B*, contenidos en el último apuntador, a su lugar correcto en el bloque *B* y se fija el bit de marca del bloque igual a 0.

Ejercicios

- 12.1 Considérese la siguiente estructura dinámica de 1000 bytes, donde los bloques en blanco están en uso, y los bloques etiquetados están enlazados en una lista libre en orden alfabético. Los números indican la posición del primer byte de cada bloque.

0	100	200	400	500	575	700	850	900	999
<i>a</i>		<i>b</i>		<i>c</i>	<i>d</i>		<i>e</i>	<i>f</i>	

Supóngase que se realizan las siguientes peticiones:

- 1) asignar un bloque de 120 bytes,
- 2) asignar un bloque de 70 bytes,
- 3) devolver al frente de la lista de disponibles el bloque que se encuentra entre los bytes 700 y 849, y
- 4) asignar un bloque de 130 bytes.

Proporcionése la lista de espacio libre, en orden, después de ejecutar la secuencia de pasos anterior, suponiendo que los bloques libres se seleccionan de acuerdo con la estrategia de

- a) primer ajuste
- b) mejor ajuste.

- 12.2 Obsérvese la siguiente estructura dinámica, donde las regiones en blanco se encuentran en uso y las regiones etiquetadas están vacías.

0	100	200	300	500
	<i>a</i>		<i>b</i>	

Establézcase una secuencia de solicitudes que puedan satisfacerse al emplear

- a) primer ajuste, pero no mejor ajuste
- b) mejor ajuste, pero no primer ajuste

- *12.3 Supóngase que se utiliza un sistema de afinidades exponencial con tamaños 1, 2, 4, 8 y 16 en una estructura dinámica de tamaño 16. Si se solicita un bloque de tamaño n , para $1 \leq n \leq 16$, es necesario asignar un bloque de tamaño 2^i , donde $2^{i-1} < n \leq 2^i$. La porción no utilizada del bloque, si existe, no puede emplearse para satisfacer ninguna otra solicitud. Si se necesita un bloque de tamaño 2^i , $i < 4$, y no existe dicho bloque libre, primero se busca un bloque de tamaño 2^{i+1} y se divide en dos partes del mismo tamaño. Si no existe un bloque de tamaño 2^{i+1} , se busca y se parte un bloque de tamaño 2^{i+2} , y así sucesivamente. Si se llega a buscar un bloque libre de tamaño 32, el proceso falla y no puede satisfacerse la solicitud. A

efectos de este ejercicio, no se combinan bloques libres adyacentes en la estructura dinámica.

Existen secuencias de solicitud $a_1, a_2, \dots, a_n$ cuya suma es menor que 16, tal que la última petición no puede satisfacerse. Por ejemplo, considérese la secuencia 5, 5, 5. La primera petición hace que el bloque inicial de tamaño 16 se parta en dos bloques de tamaño 8, uno de los cuales se usa para satisfacer la solicitud. El bloque libre restante de tamaño 8 satisface la segunda, y no queda espacio libre para satisfacer la tercera.

Encuéntrese una secuencia $a_1, a_2, \dots, a_n$ de enteros entre 1 y 16 (no es necesario que sean idénticos), cuya suma sea lo más pequeña posible, tal que, tratada como una secuencia de solicitudes de bloques de tamaño $a_1, a_2, \dots, a_n$, la última petición no se satisfaga. Explíquese por qué esta secuencia de peticiones no se satisface, pero cualquier otra secuencia cuya suma sea más pequeña sí puede satisfacerse.

- 12.4 Considérese la compactación de memoria en la administración de bloques de igual tamaño. Supóngase que cada bloque consta de un campo para datos y otro para apuntador, y que se han marcado todos los bloques que se encuentran en uso actualmente. Los bloques están ubicados entre las localidades de memoria a y b . Se desea relocalizar todos los bloques activos de manera que ocupen memoria contigua partiendo de a . Recuérdese que, para relocalizar un bloque, el campo del apuntador de cualquier bloque que apunta al bloque relocalizado debe actualizarse. Diseñese un algoritmo para la compactación de bloques.
- 12.5 Se da un arreglo de tamaño n . Proporcionese un algoritmo para correr todos los elementos del arreglo k lugares en forma cíclica, en sentido contrario al de las manecillas del reloj, sólo con una cantidad constante de memoria adicional, independiente de k y n . *Sugerencia.* Téngase en cuenta qué sucede si se invierten los k primeros elementos, los $n-k$ últimos elementos y, por último, el arreglo completo.
- 12.6 Diseñese un algoritmo para sustituir una subcadena y de una cadena xyz por otra subcadena y' , con la menor cantidad posible de memoria adicional. ¿Cuál es la complejidad de tiempo y espacio de este algoritmo?
- 12.7 Escribese un programa para hacer una copia de una lista dada. ¿Cuál es la complejidad de tiempo y espacio del programa?
- 12.8 Escribese un programa para determinar si dos listas son idénticas. ¿Cuál es la complejidad de tiempo y espacio de este programa?
- 12.9 Obtégase el algoritmo de compactación de las estructuras dinámicas de Morris, mostrado en la sección 12.6.
- *12.10 Diseñese un esquema de asignación de almacenamiento para una situación en la que la memoria se asigna y libera en bloques de longitud 1 y 2. Proporcionense cotas sobre la eficiencia del algoritmo.

Notas bibliográficas

La administración eficiente almacenamiento es un tema central en muchos lenguajes de programación, incluyendo SNOBOL [Farber, Griswold y Polonsky (1964)], LISP [McCarthy (1965)], APL [Iverson (1962)] y SETL [Schwartz (1973)]. Nicholls [1975] y Pratt [1975] comentan las técnicas de administración de almacenamiento en el contexto de la compilación de lenguajes de programación.

El sistema de afinidades para asignación de almacenamiento fue publicado por primera vez por Knowlton [1965]. Los sistemas de afinidades de Fibonacci fueron estudiados por Hirschberg [1973].

El elegante algoritmo de marcado utilizado en la recolección de basura fue descubierto por Peter Deutsch (Deutsch y Bobrow [1966]) y por Schorr y Waite [1965]. El esquema de compactación de estructuras dinámicas de la sección 12.6 se debe a Morris [1978].

Robson [1971] y Robson [1974] analizan la cantidad de memoria requerida para algoritmos dinámicos de asignación de almacenamiento. Robson [1977] presenta un algoritmo acotado en espacio de trabajo para copiar estructuras cíclicas. Fletcher y Silver [1966] contiene otra solución al ejercicio 12.5 que utiliza poca memoria adicional.